Machine Learning Fundamentals

From "what is ML?" to training neural networks

Classification · Regression · Metrics · Loss · Neural Networks · Backprop · Optimizers

Satendra Kumar

Our Journey Today

1
What is Machine Learning?

2
Regression — predicting numbers

3
Loss functions — measuring "how wrong"

4
Classification — binary, multiclass, multilabel

5
Evaluating models — accuracy, precision, recall, F1

6
Neural networks — neurons, hidden layers, activations (ReLU, sigmoid, softmax), parameters

7
Feedforward & backpropagation

8
Optimization — SGD, Momentum, RMSProp, Adam

Each idea builds on the previous one. Colour code used throughout: input / data · weights · hidden · output · error / loss

01

What is Machine Learning?

🤔 What do you think?

So, What Is Machine Learning?

Machine learning is teaching computers to **learn from examples**, instead of programming every rule by hand.

So, What Is Machine Learning?

Machine learning is teaching computers to **learn from examples**, instead of programming every rule by hand.

📚 Learn

Look at lots of **past examples**, e.g. transactions marked fraud or genuine

🔍 Find patterns

Work out the **rules** that connect inputs to answers. These rules are the **model**

🎯 Predict

Use those rules to give **answers** for new data it has never seen

So, What Is Machine Learning?

Machine learning is teaching computers to **learn from examples**, instead of programming every rule by hand.

📚 Learn

Look at lots of **past examples**, e.g. transactions marked fraud or genuine

🔍 Find patterns

Work out the **rules** that connect inputs to answers. These rules are the **model**

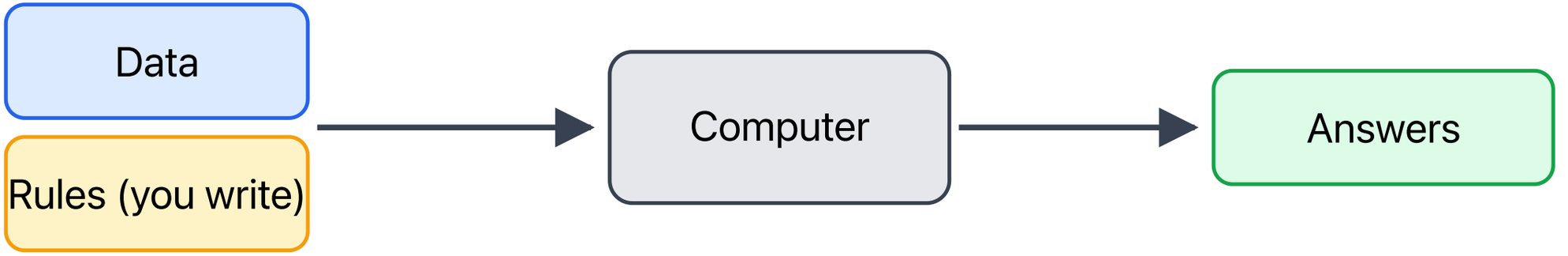
🎯 Predict

Use those rules to give **answers** for new data it has never seen

> "The field of study that gives computers the ability to learn without being explicitly programmed." — Arthur Samuel, 1959

Traditional Programming vs Machine Learning

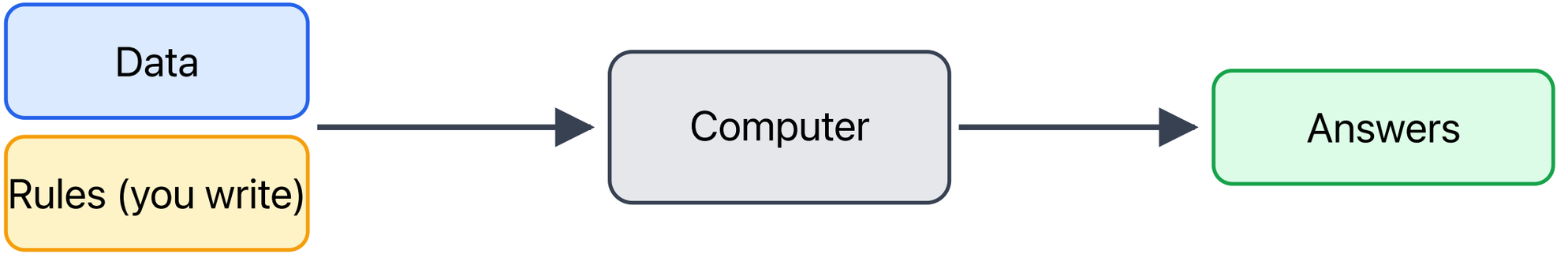
Traditional programming



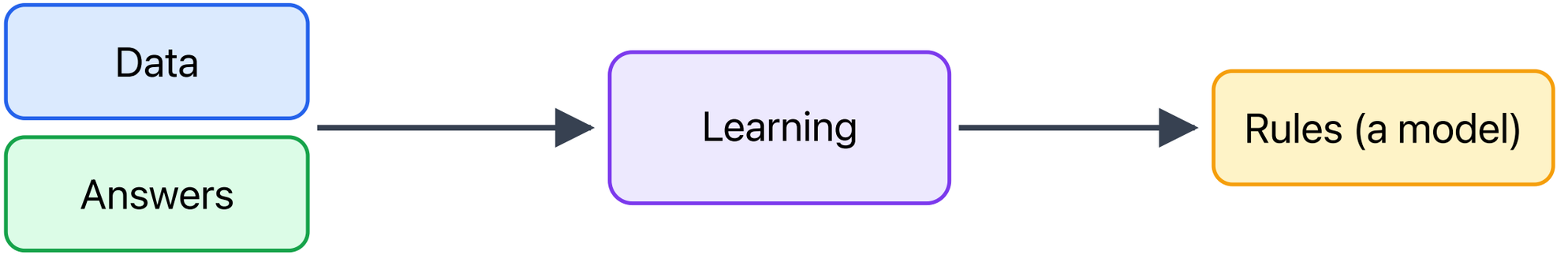
In plain English: Instead of writing the rules ourselves, we show the computer many examples with answers, and it **figures out the rules** on its own.

Traditional Programming vs Machine Learning

Traditional programming



Machine learning



In plain English: Instead of writing the rules ourselves, we show the computer many examples with answers, and it **figures out the rules** on its own.

ML Is Everywhere

🏦 Banking
Fraud detection on card transactions, credit scoring, loan default prediction

📧 Email
Spam filtering, smart replies, auto-sorting into folders

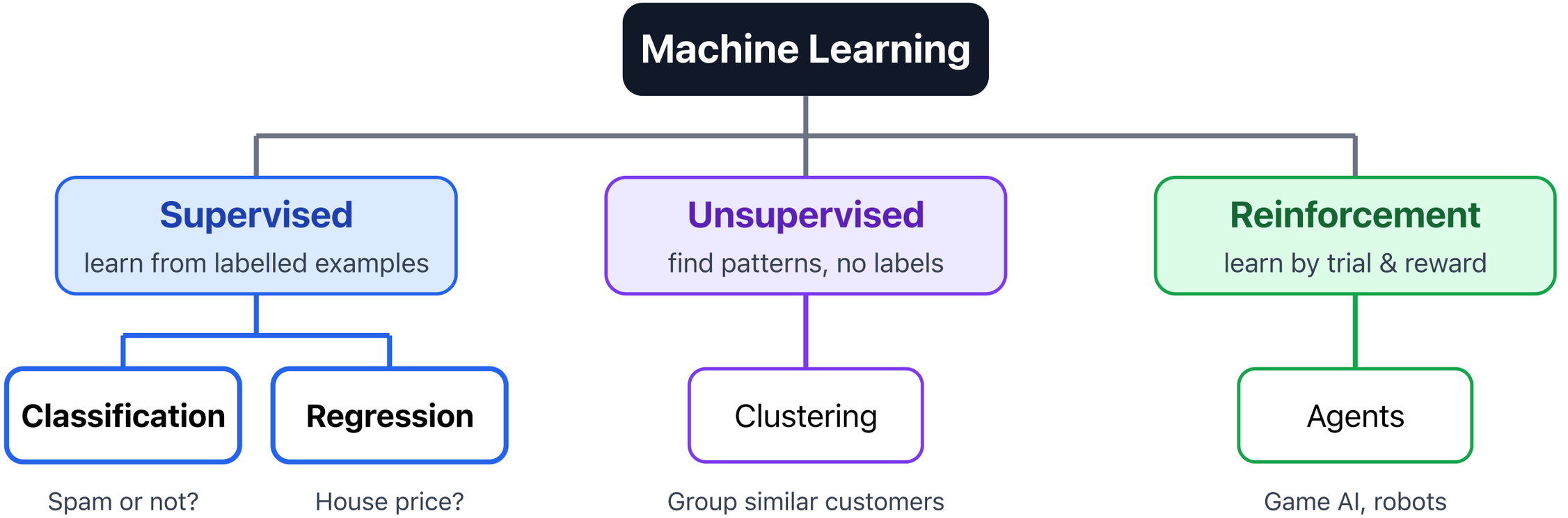
🎬 Streaming
"Because you watched…" recommendations

📱 Phones
Face unlock, voice assistants, keyboard autocomplete

🗺️ Maps
Arrival-time prediction, traffic forecasts

🏥 Health
Reading X-rays, predicting patient risk

Three Main Types of Machine Learning



⭐ **Today's focus: supervised learning**

The Vocabulary: Features and Labels

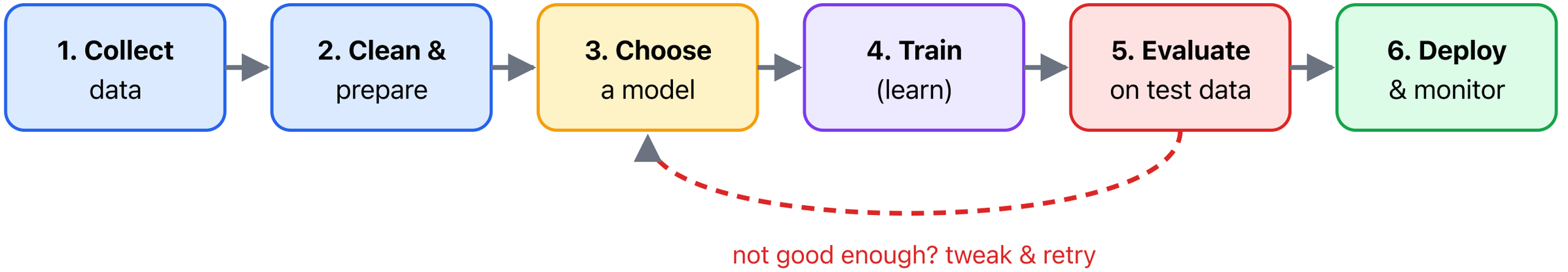
Example: predicting the price of a house

Area (sq ft)	Bedrooms	Age (years)	Price (₹ lakh)
1000	2	5	55
1500	3	2	90
800	1	15	35
1200	3	10	?

Features (x) — the inputs we know. Also called attributes or variables.

In plain English: Label (y) is the answer we want the model to predict. Each row is one **example** (also called a sample).

The Machine Learning Workflow



We split data into a **training set** (to learn from, ~80%) and a **test set** (to check on unseen data, ~20%) — like practice questions vs. the final exam.

02

Regression

Predicting a number

What Is Regression?

Regression answers **"How much?"** or **"How many?"** — the output is a **continuous number**.

🏠
House price from area, location, age

💰
Customer's monthly spending

🌡️
Tomorrow's temperature

📈
Next month's sales

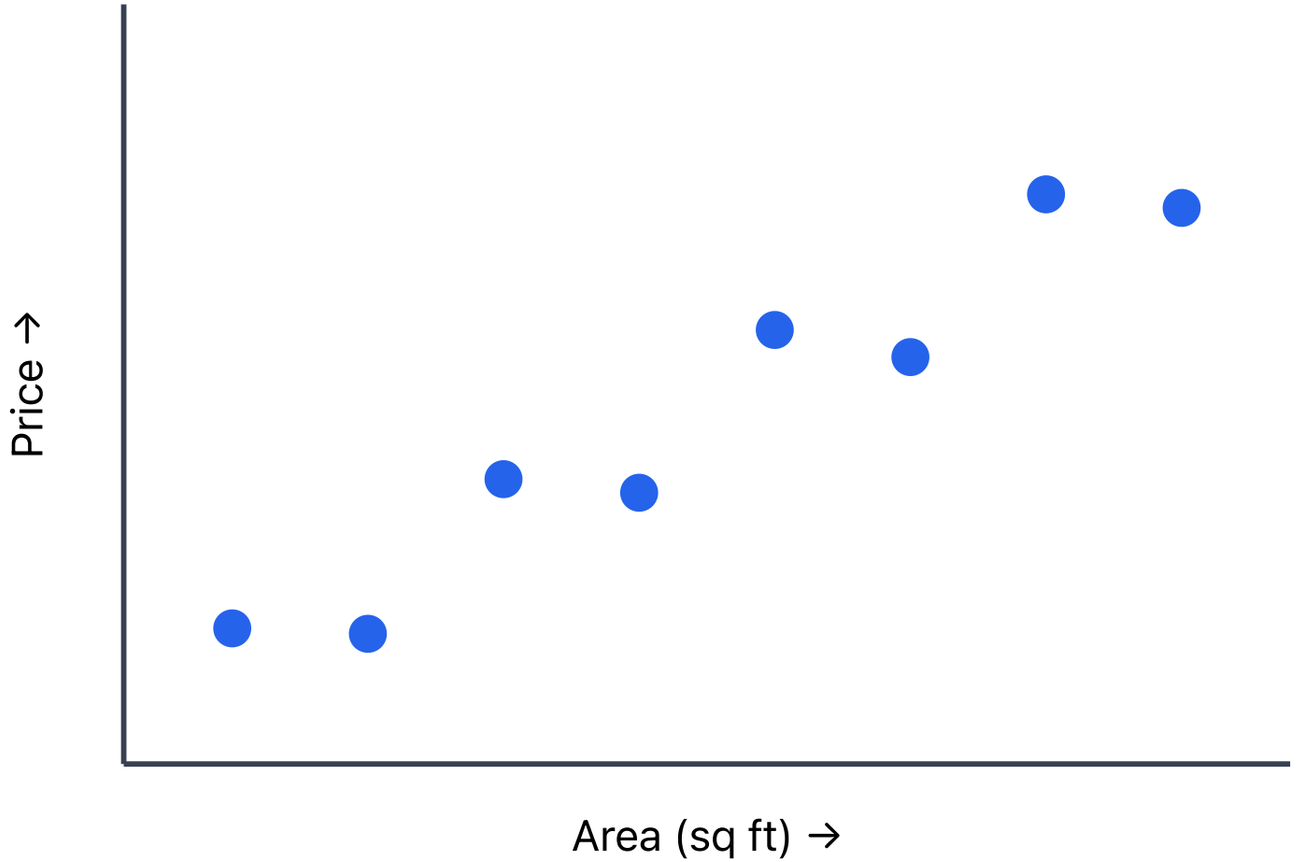
In plain English: If the answer can be 54.3, 54.31 or 54.312 — any value on a number line — it's a regression problem.

Linear Regression: Fit a Straight Line

$$\hat{y} = w\,x + b$$

- x — input (area)
- $\hat{y}$ — prediction ("y-hat")
- w — **weight** / slope: how much price rises per sq ft
- b — **bias** / intercept: base price

> **In plain English:** Learning = finding the **w** and **b** that make the line pass as close as possible to all the dots.

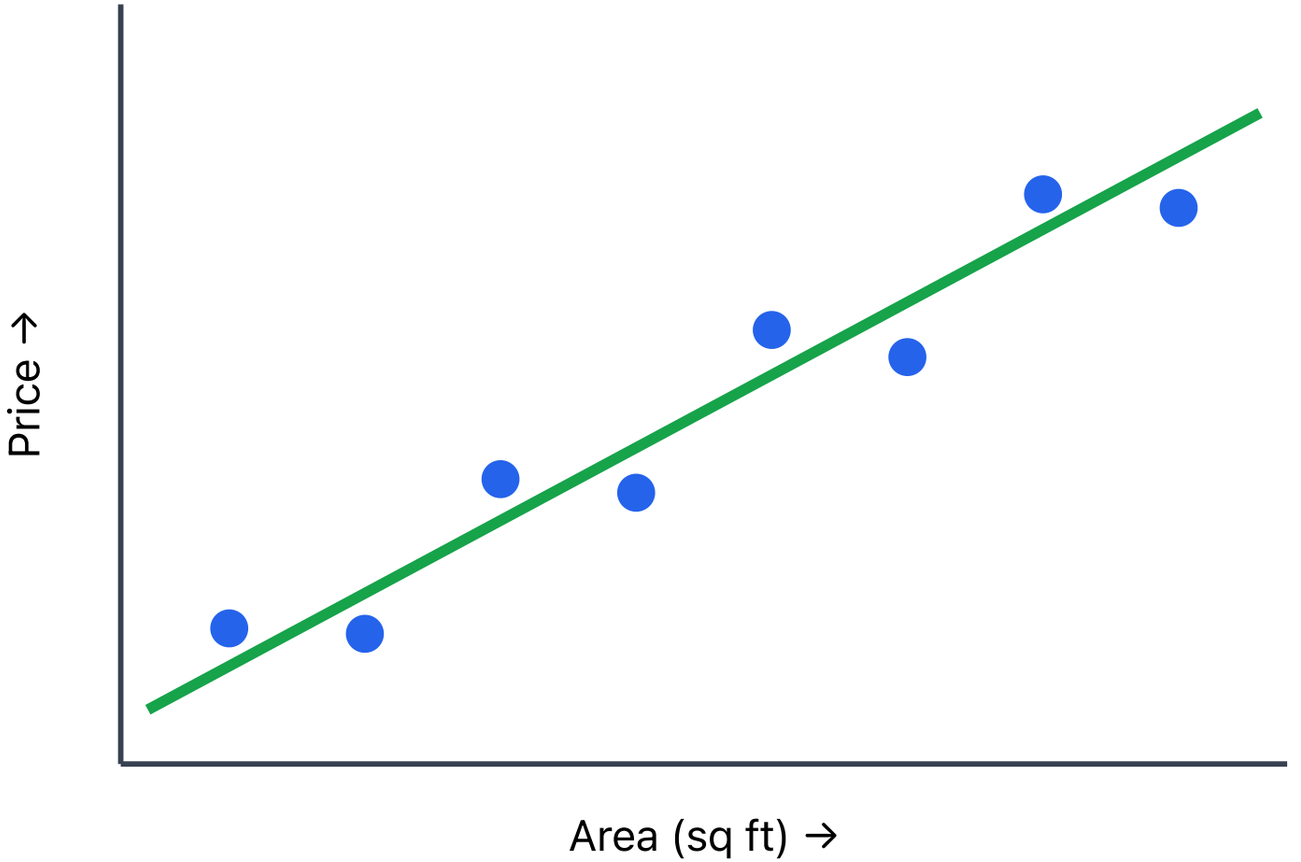


Linear Regression: Fit a Straight Line

$$\hat{y} = w\,x + b$$

- x — input (area)
- $\hat{y}$ — prediction ("y-hat")
- w — **weight** / slope: how much price rises per sq ft
- b — **bias** / intercept: base price

> **In plain English:** Learning = finding the **w** and **b** that make the line pass as close as possible to all the dots.



Worked Example

Suppose training found: $w = 0.05$ lakh per sq ft and $b = 10$ lakh.

$$\hat{y} = 0.05 \times 1000 + 10 = 60 \text{ lakh}$$

A 1000 sq ft house → predicted price ₹60 lakh.

Worked Example

Suppose training found: $w = 0.05$ lakh per sq ft and $b = 10$ lakh.

$$\hat{y} = 0.05 \times 1000 + 10 = 60 \text{ lakh}$$

A 1000 sq ft house → predicted price ₹60 lakh.

More than one feature? Just add more terms

$$\hat{y} = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

$$\hat{y} = 0.05 \cdot \underbrace{1200}_{\text{area}} + 5 \cdot \underbrace{3}_{\text{bedrooms}} - 0.8 \cdot \underbrace{10}_{\text{age}} + 10 = 60 + 15 - 8 + 10 = 77 \text{ lakh}$$

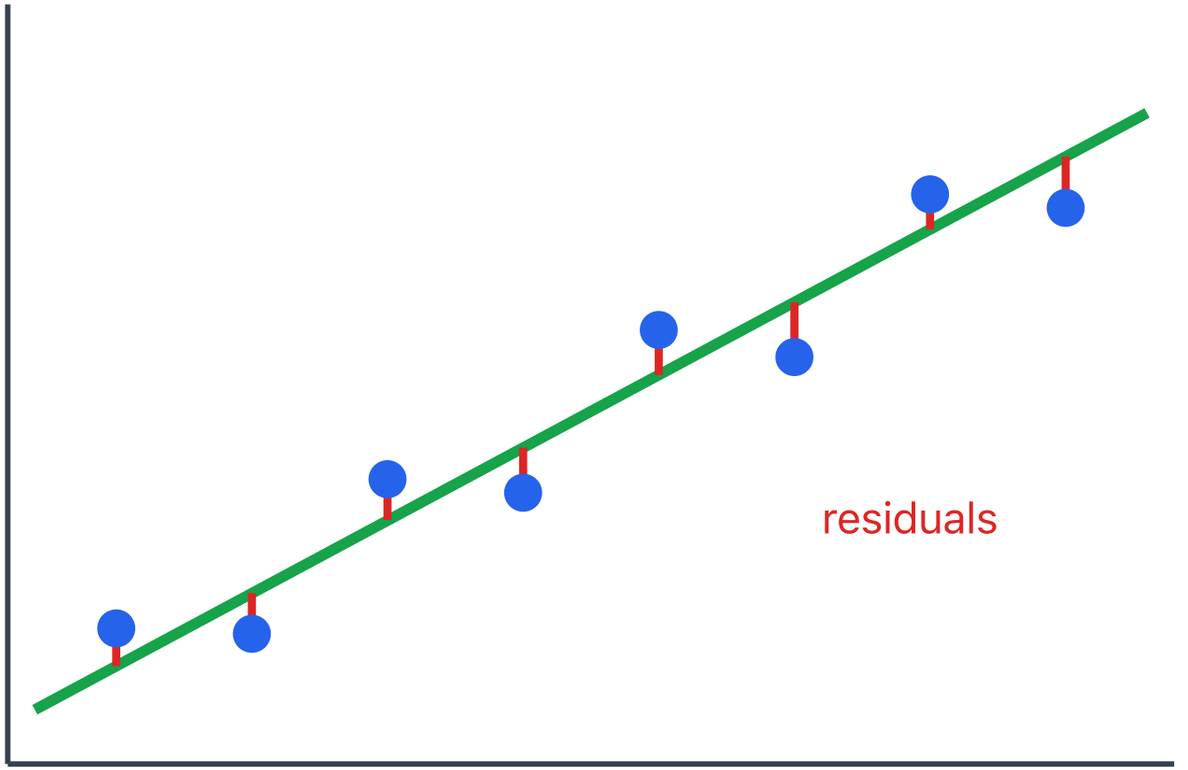
In plain English: Each feature gets its own weight. A negative weight (age) means "more of this → lower price".

How Wrong Is Our Line? Residuals

For each point, the **error (residual)** is the gap between the real value and the prediction:

$$\text{error}_i = y_i - \hat{y}_i$$

> **In plain English:** The red lines show how far off we are. A good model makes these lines **short**. To compare models we need **one number** that sums up all the red lines — that's a **loss function**.



03

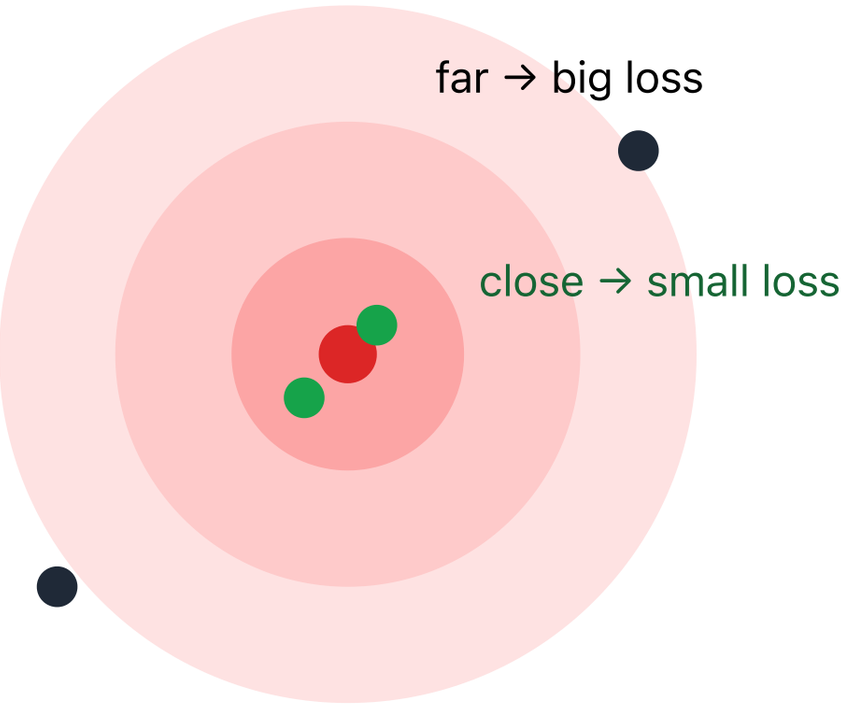
Loss Functions

One number that says "how wrong" the model is

What Is a Loss Function?

- A formula that compares **predictions** with **true answers**
- Outputs a single number: the **loss**
- **Lower = better.** Perfect predictions → loss of 0
- Training = **adjusting weights to make the loss smaller**

> **In plain English:** The loss is the model's "score" in a game where the goal is to get as low as possible — like golf.



Mean Squared Error (MSE) — for Regression

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2$$

take each error → **square** it → **average** them

House	Actual y	Predicted $\hat{y}$	Error	Error²
A	200	210	−10	100
B	300	280	+20	400
C	400	400	0	0

$$\text{MSE} = \frac{100 + 400 + 0}{3} \approx 166.7$$

In plain English: Squaring makes every error positive (so −10 and +10 don't cancel) and punishes big mistakes much more than small ones.

MSE vs MAE — How They Treat Outliers

$$\text{MAE} = \frac{1}{n}\sum |y_i - \hat{y}_i|$$

Mean Absolute Error just averages the size of the errors (no squaring).

Errors: **1, 1, 1, 10** (one outlier)

- MAE $= (1+1+1+10)/4 = 3.25$
- MSE $= (1+1+1+100)/4 = 25.75$

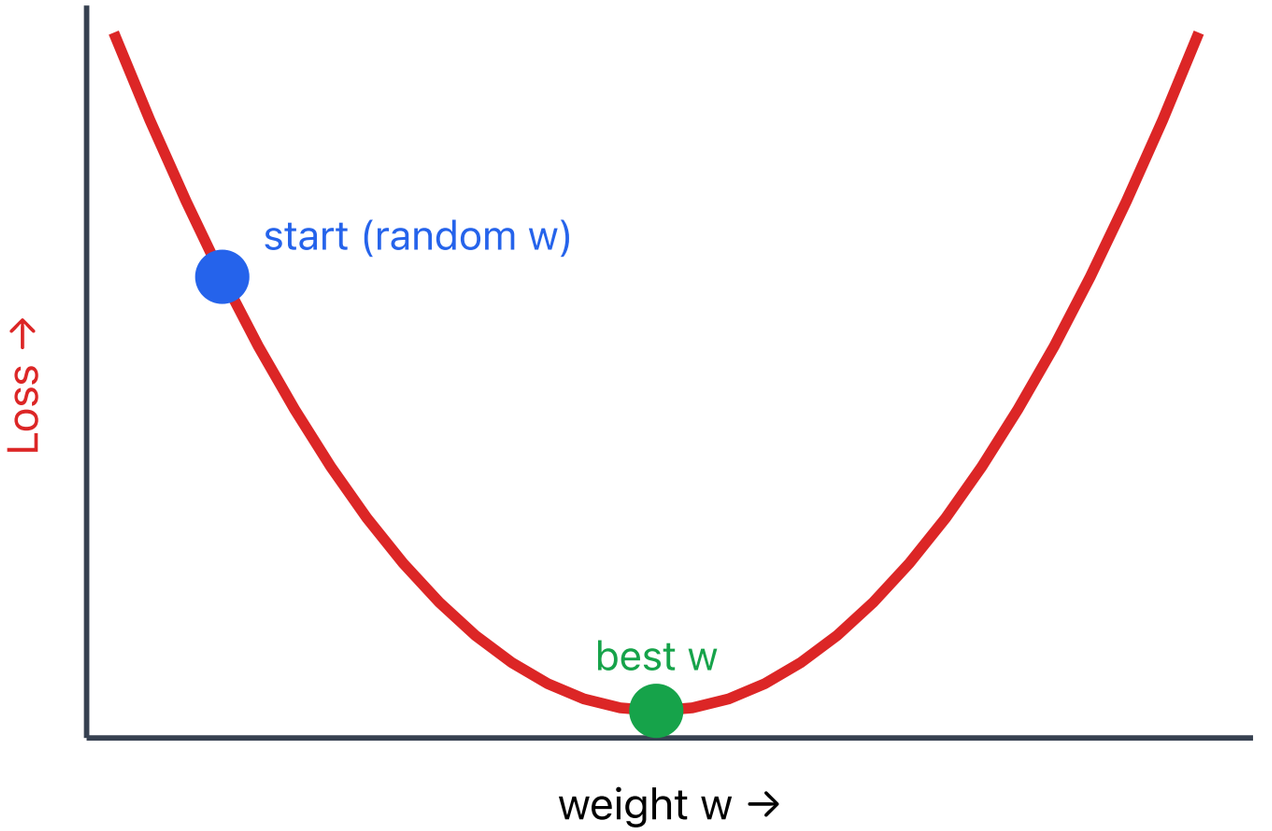
	MSE	MAE
Big errors	punished heavily	treated linearly
Outliers	very sensitive	robust
Units	squared (₹²)	same as y (₹)
Smooth for maths	✅ yes	kink at 0

RMSE $= \sqrt{\text{MSE}}$ brings the units back to the original scale — easier to explain: "we're off by about ₹12.9 lakh on average".

The Loss Landscape

If we try every possible weight w and plot the loss, MSE gives a nice **bowl** shape.

> **In plain English:** Training = rolling down to the **bottom of the bowl**, where the loss is smallest. How we roll down is called **optimization** (section 8).



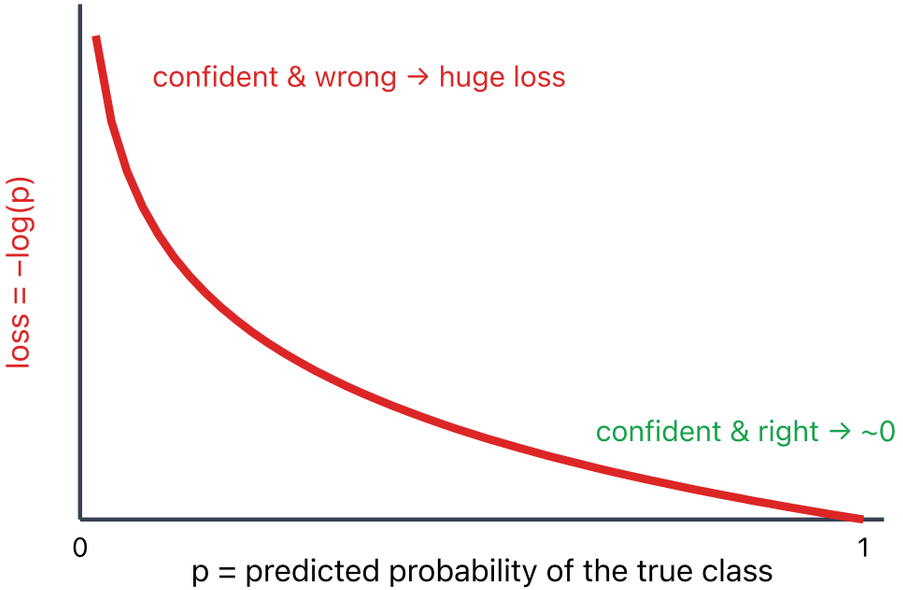
Cross-Entropy — for Classification

When the model outputs a **probability** p that the answer is "yes", we use **Binary Cross-Entropy (log loss)**:

$$L = -\big[\, y \log(p) + (1 - y) \log(1 - p) \,\big]$$

If the true answer is yes ($y = 1$), this simplifies to $L = -\log(p)$:

True	Model says p(yes)	Loss
Yes	0.9 (confident, right)	0.11
Yes	0.5 (unsure)	0.69
Yes	0.1 (confident, wrong)	2.30



In plain English: Cross-entropy rewards being confident and right, and **heavily punishes being confident and wrong**.

04

Classification

Predicting a category

What Is Classification?

Classification answers **"Which one?"** — the output is a **category (class)**.



Spam or not spam?



Fraud or genuine transaction?



Approve or reject loan?



Cat, dog or rabbit?

Regression: "What will the house cost?" → ₹77 lakh

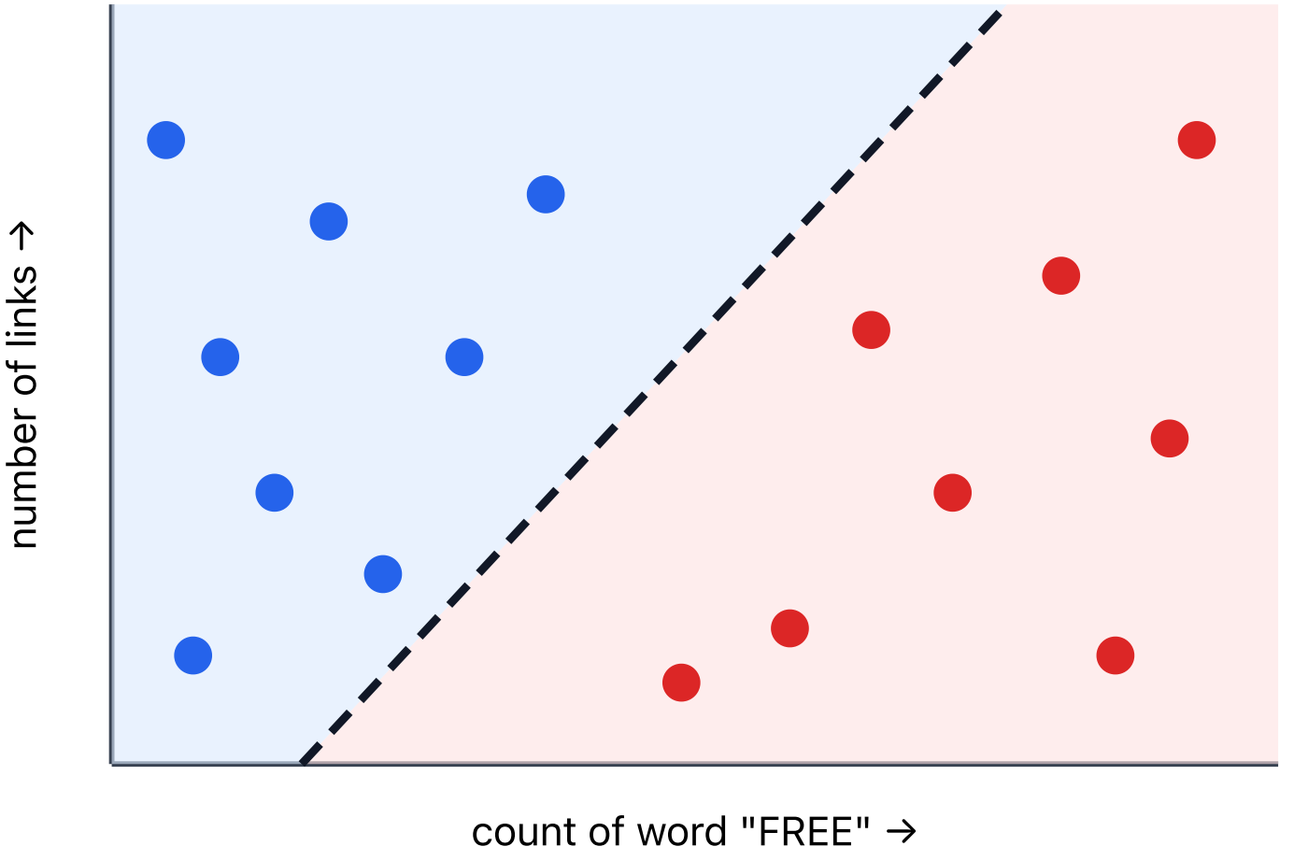
Classification: "Will the house sell this month?" → Yes / No

The Decision Boundary

Plot emails by two features. The model learns a **line (boundary)** that separates the classes.

- Spam
- Not spam

> **In plain English:** A new email lands on one side of the line → that's its predicted class.

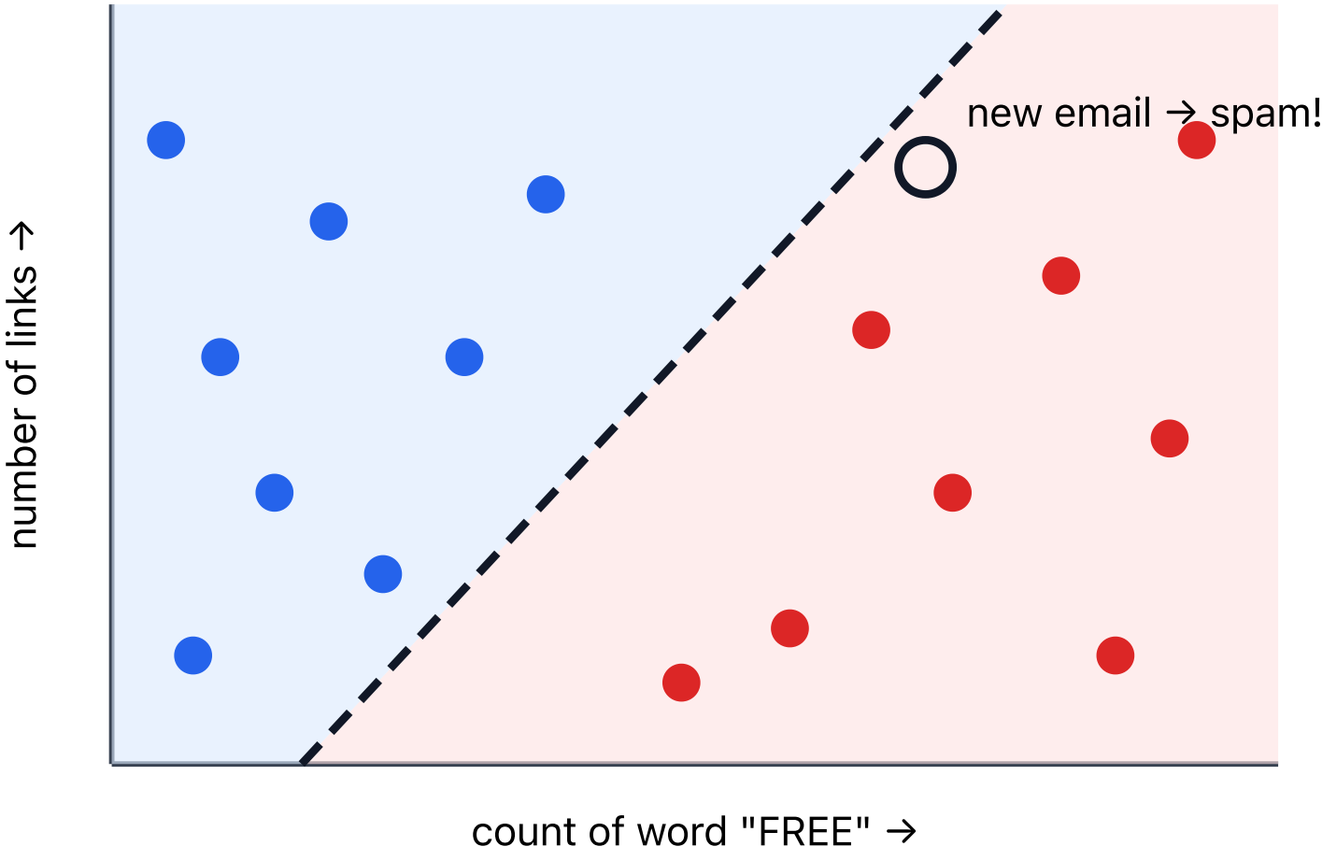


The Decision Boundary

Plot emails by two features. The model learns a **line (boundary)** that separates the classes.

- Spam
- Not spam

> **In plain English:** A new email lands on one side of the line → that's its predicted class.



Logistic Regression: Turning Scores into Probabilities

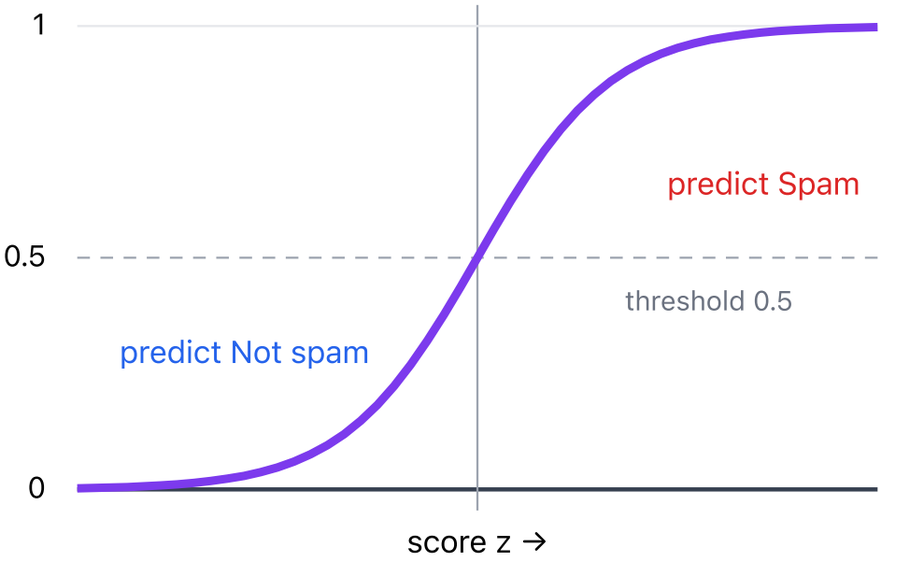
Step 1 — compute a score, exactly like linear regression:

$$z = w_1 x_1 + w_2 x_2 + b$$

Step 2 — squash it into a probability between 0 and 1 using the **sigmoid**:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Step 3 — decide: if $p \geq 0.5$ → **Spam**, else **Not spam**



Worked Example: Is This Email Spam?

Learned weights: $w_1 = 1.5$ (per "FREE"), $w_2 = 0.8$ (has link), $b = -2$

New email: the word "FREE" appears 2 times and it contains a link (1).

$$z = 1.5 \times 2 + 0.8 \times 1 + (-2) = 3 + 0.8 - 2 = 1.8$$

Worked Example: Is This Email Spam?

Learned weights: $w_1 = 1.5$ (per "FREE"), $w_2 = 0.8$ (has link), $b = -2$

New email: the word "FREE" appears 2 times and it contains a link (1).

$$z = 1.5 \times 2 + 0.8 \times 1 + (-2) = 3 + 0.8 - 2 = 1.8$$

$$p = \sigma(1.8) = \frac{1}{1 + e^{-1.8}} = \frac{1}{1 + 0.165} \approx 0.86$$

Worked Example: Is This Email Spam?

Learned weights: $w_1 = 1.5$ (per "FREE"), $w_2 = 0.8$ (has link), $b = -2$

New email: the word "FREE" appears 2 times and it contains a link (1).

$$z = 1.5 \times 2 + 0.8 \times 1 + (-2) = 3 + 0.8 - 2 = 1.8$$

$$p = \sigma(1.8) = \frac{1}{1 + e^{-1.8}} = \frac{1}{1 + 0.165} \approx 0.86$$

In plain English: The model is 86% sure this email is spam. 0.86 ≥ 0.5, so it goes to the spam folder.

Three Flavours of Classification

Binary

2 classes, pick **one**

📧 Spam / Not spam

💳 Fraud / Genuine

Spam ✓ | Not spam

Multiclass

3+ classes, pick exactly **one**

🐱 Cat / Dog / Rabbit

✍️ Digit 0–9

Cat ✓ | Dog | Rabbit

Multilabel

Many labels, pick **any number**

🎬 Movie genres

📷 Photo tags: beach, sunset, people

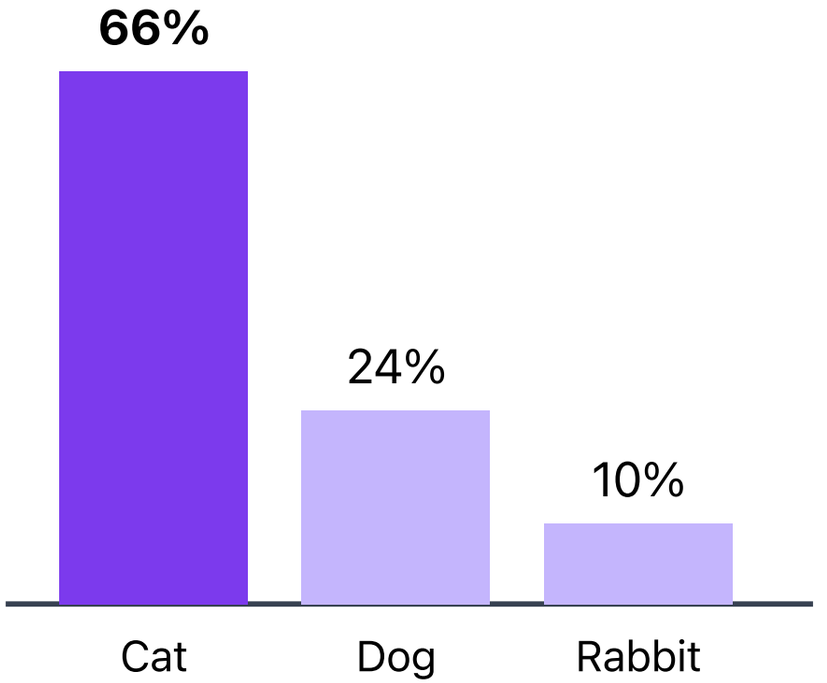
Action✓ | Drama | Sci-Fi✓

Multiclass: The Softmax Function

The model produces one raw score z_k per class. **Softmax** turns them into probabilities that **add up to 1**:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

Class	Score z	e^z	Probability
🐱 Cat	2.0	7.39	**0.66**
🐶 Dog	1.0	2.72	0.24
🐰 Rabbit	0.1	1.11	0.10
		sum = 11.21	sum = 1.00



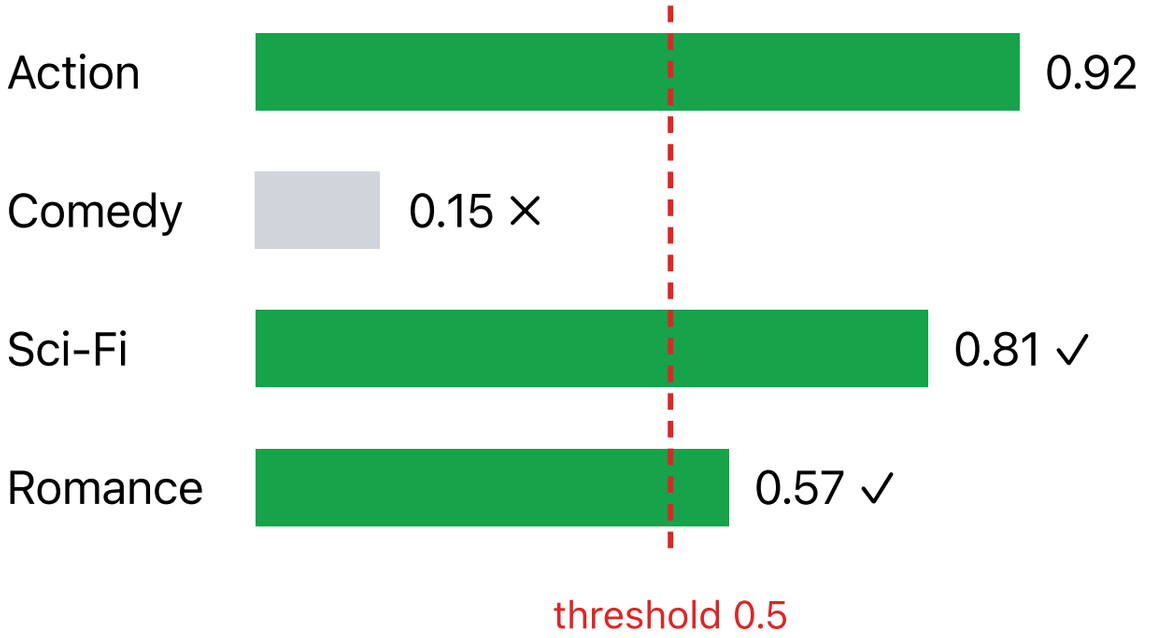
In plain English: Softmax is a "competition" — classes share 100% of the probability. Pick the class with the highest share: **Cat**.

Multilabel: Independent Yes/No for Each Label

Each label gets its **own sigmoid** — a separate yes/no question.
Probabilities do **not** need to sum to 1.

$$p_k = \sigma(z_k) = \frac{1}{1 + e^{-z_k}} \quad \text{for each label } k$$

Movie: "Interstellar" (threshold 0.5)



In plain English: The movie is tagged Action + Sci-Fi + Romance. The sum is 2.45 — that's fine, because each label is decided independently.

Cheat Sheet: Binary vs Multiclass vs Multilabel

	Binary	Multiclass	Multilabel
Question	Yes or no?	Which one of K?	Which of K apply?
Answers per example	exactly 1 of 2	exactly 1 of K	0 to K
Output neurons	1	K	K
Output activation	sigmoid	softmax	sigmoid (each)
Loss function	binary cross-entropy	categorical cross-entropy	binary cross-entropy per label
Example	Fraud?	Digit 0–9	Photo tags

Categorical cross-entropy: $L = -\sum_k y_k \log(p_k) = -\log(p_{\text{true class}})$. For the cat example: $-\log(0.66) \approx 0.42$.

05

Evaluating a Classifier

Accuracy, confusion matrix, precision, recall, F1

The Accuracy Trap 🪤

A bank has **1,000** card transactions. Only **10** are fraud.

A "lazy" model predicts **"Genuine"** for every single transaction.

$$\text{Accuracy} = \frac{990 \text{ correct}}{1000} = 99\%$$

The Accuracy Trap 

A bank has **1,000** card transactions. Only **10** are fraud.

A "lazy" model predicts **"Genuine"** for every single transaction.

$$\text{Accuracy} = \frac{990 \text{ correct}}{1000} = 99\%$$

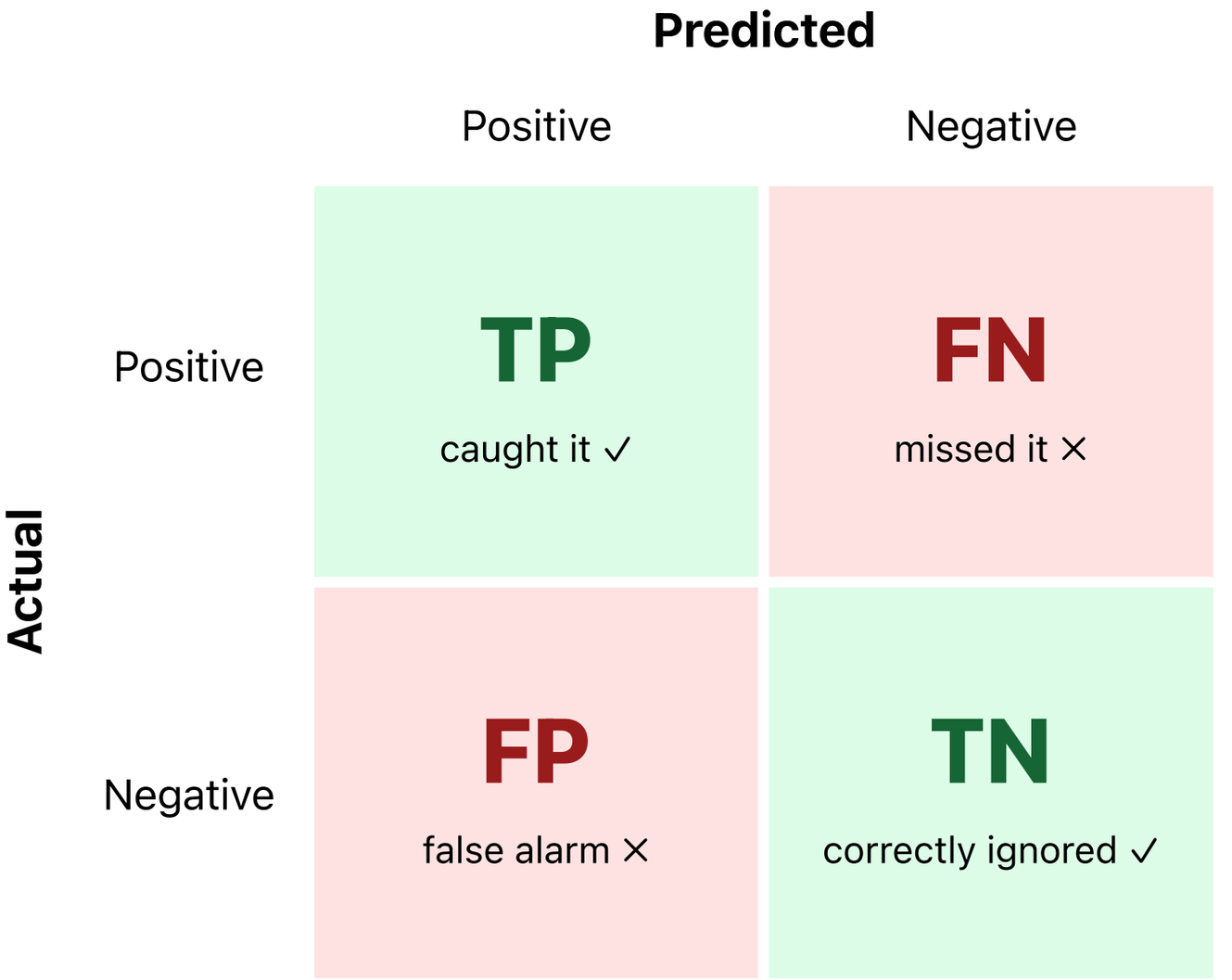
> 99% accurate... yet it catches **zero** fraud cases! When classes are **imbalanced**, accuracy alone is misleading. We need to look at which kinds of mistakes the model makes.

The Confusion Matrix

A table that counts the 4 possible outcomes:

- **TP** — True Positive: said yes, was yes ✅
- **TN** — True Negative: said no, was no ✅
- **FP** — False Positive: said yes, was no ❌ "false alarm"
- **FN** — False Negative: said no, was yes ❌ "missed it"

> **In plain English:** The first word (True/False) tells you if the model was right. The second word is what the model **predicted**.



Real Example: A COVID Test on 100 People

Really \ Test says	Positive	Negative
Sick (10)	**8** TP	**2** FN
Healthy (90)	**5** FP	**85** TN

- 10 people are really sick, 90 are healthy
- Test caught **8** sick people (TP)
- Test missed **2** sick people (FN)
- Test wrongly flagged **5** healthy people (FP)
- Test correctly cleared **85** healthy people (TN)

> We'll compute every metric from these 4 numbers.

Accuracy & Precision

Accuracy

$$\frac{TP + TN}{TP + TN + FP + FN} = \frac{8 + 85}{100} = 93\%$$

In plain English: Out of everyone, how many did the test get right?

Precision

$$\frac{TP}{TP + FP} = \frac{8}{8 + 5} = 61.5\%$$

In plain English: When the test says "sick", how often is it right? Only ~6 out of 10 positive results are real.

Precision focuses on the **Predicted Positive column**: "Can I trust a positive result?" High precision = few false alarms.

Recall & F1 Score

Recall (Sensitivity)

$$\frac{TP}{TP + FN} = \frac{8}{8 + 2} = 80\%$$

In plain English: Out of all truly sick people, how many did we catch? We found 8 of 10.

F1 Score

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R} = 2 \cdot \frac{0.615 \times 0.8}{0.615 + 0.8} = 69.6\%$$

In plain English: A single score that balances precision and recall. It's high only when **both** are high.

Why not a simple average? If P = 100% and R = 1%, average = 50.5% (looks OK!) but F1 = **2%** — the harmonic mean exposes the weak side.

Precision or Recall? It Depends on the Cost of Mistakes

Favour Precision when false alarms hurt

- 📧 **Spam filter** — sending your job offer to spam is worse than letting one spam through
- 🔍 **Search results** — top results must be relevant
- ⚖️ **Auto-blocking a customer account** — don't lock out genuine users

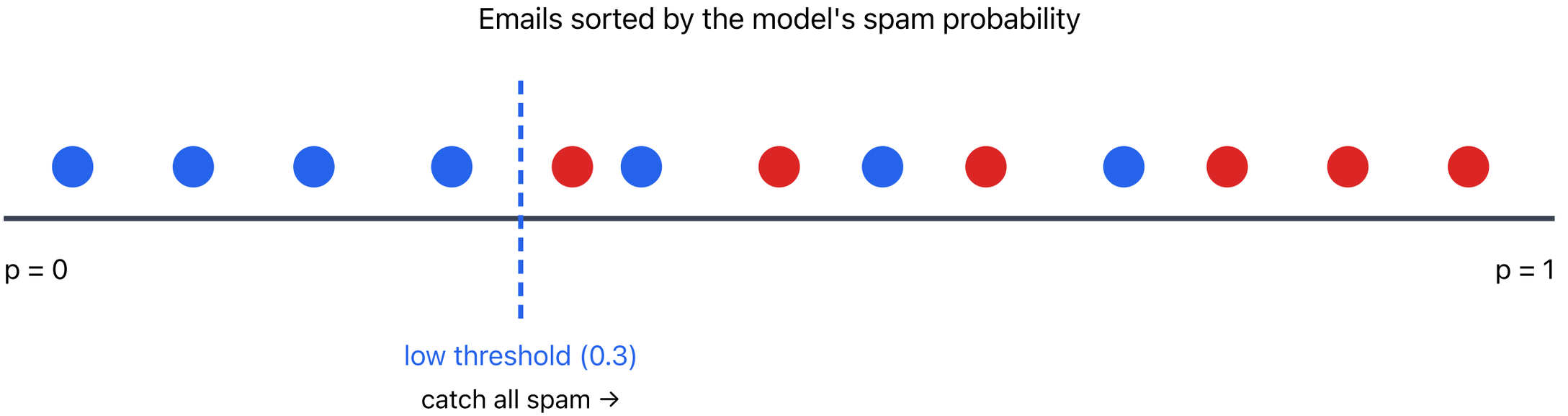
Favour Recall when misses hurt

- 🏥 **Cancer screening** — missing a patient is dangerous; extra tests are OK
- 💳 **Fraud detection** — missing fraud costs money; a quick verification call is cheap
- ✈️ **Airport security** — better to check a few extra bags

In plain English: Ask: "Which is worse — a false alarm (FP) or a miss (FN)?" Use **F1** when both matter.

The Precision–Recall Trade-off

Moving the decision threshold changes which emails are flagged:



high recall, more false alarms → lower precision high threshold (0.8) only flag sure cases → **high precision**, miss some spam → lower recall spam not spam

> **In plain English:** You can't usually max out both. Raising the threshold trades recall for precision, and lowering it does the opposite.

Confusion Matrix for Multiclass

Actual ↓ / Predicted →	🐱 Cat	🐶 Dog	🐰 Rabbit
🐱 Cat	**8**	1	1
🐶 Dog	2	**7**	1
🐰 Rabbit	0	1	**9**

- Diagonal = correct. Accuracy $= 24/30 = 80\%$
- Precision (Cat) $= 8/(8+2+0) = 80\%$
- Recall (Cat) $= 8/(8+1+1) = 80\%$
- Recall (Dog) $= 7/10 = 70\%$ — dogs are often confused with cats

In plain English: Compute precision/recall for each class (treat it as "this class vs the rest"), then average them. A **macro average** weights every class equally.

Metrics Summary

Metric	Formula	Question it answers	COVID example
Accuracy	$\frac{TP+TN}{\text{all}}$	How often is the model right overall?	93%
Precision	$\frac{TP}{TP+FP}$	Can I trust a "yes"?	61.5%
Recall	$\frac{TP}{TP+FN}$	Did we find all the real "yes" cases?	80%
F1	$\frac{2PR}{P+R}$	Balance of precision & recall	69.6%

> Remember: **P**recision = **P**redicted positives are correct; **R**ecall = **R**eal positives are found.

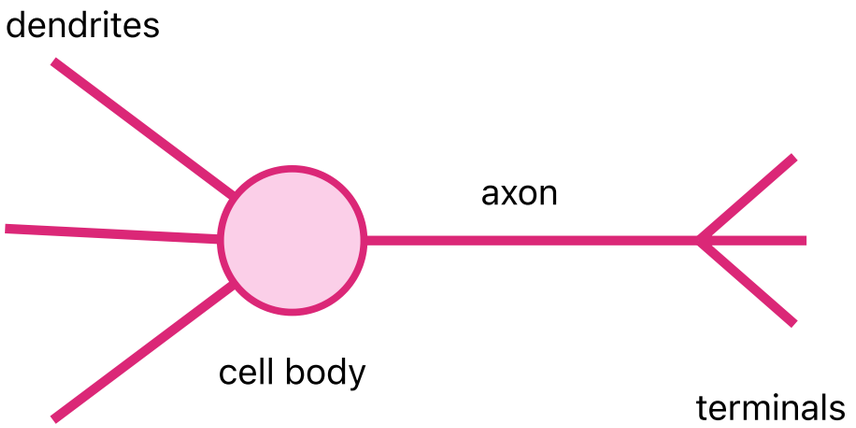
06

Neural Networks

Neurons, perceptrons, hidden layers, activation functions, parameters & tuning

Inspired by the Brain

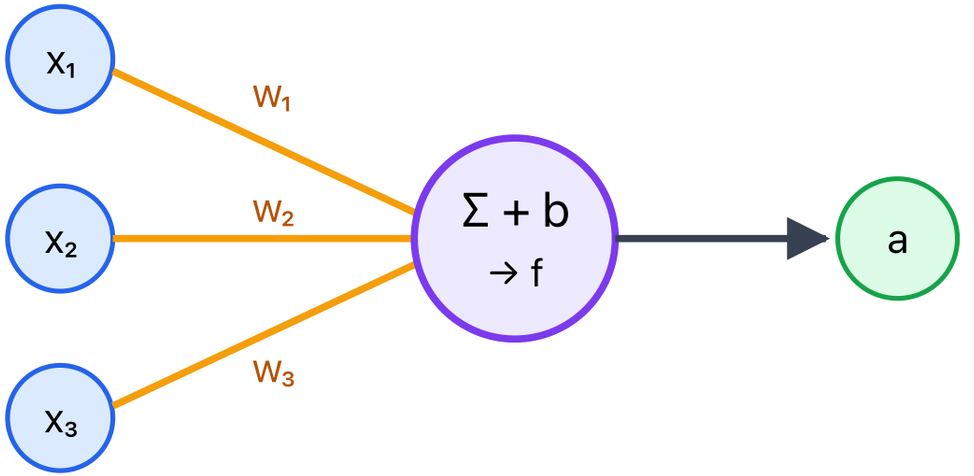
Biological neuron



receives signals → if strong enough, "fires"

≈

Artificial neuron



inputs weights sum + activate output

What a Single Neuron Computes

$$z = \sum_{i=1}^{n} w_i\, x_i + b \qquad\qquad a = f(z)$$

1. **Multiply** each input by its weight (how important is it?)
2. **Add** them up, plus a bias (a baseline nudge)
3. **Pass** through an activation function f (decide how strongly to "fire")

Example: $x = (2, 3)$, $w = (0.5, -1)$, $b = 1$: $z = 0.5(2) + (-1)(3) + 1 = -1$, and with ReLU $a = \max(0, -1) = 0$ → the neuron stays quiet.

> **In plain English:** A neuron is just a weighted vote followed by a decision. Linear regression is a neuron with no activation function!

The Perceptron (1958) — The First Neuron

Uses a **step function**: output is 1 if $z \geq 0$, else 0.

$$\hat{y} = \begin{cases} 1 & \text{if } w_1 x_1 + w_2 x_2 + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Learning rule: for each mistake, nudge the weights toward the right answer:

$$w_i \leftarrow w_i + \eta \, (y - \hat{y}) \, x_i$$

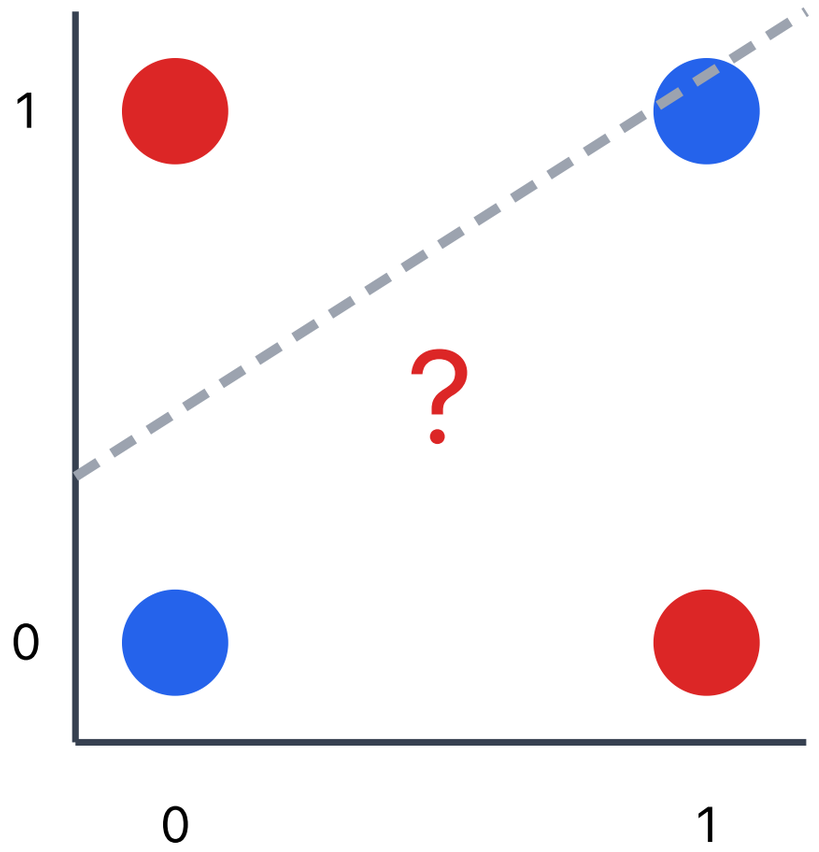
η (eta) = learning rate, a small step size like 0.1

Example: an AND gate with $w_1 = w_2 = 1,\ b = -1.5$

x_1	x_2	z	output
0	0	−1.5	0
0	1	−0.5	0
1	0	−0.5	0
1	1	+0.5	1

> **In plain English:** Fires only when both inputs are on.

The Perceptron's Limit: XOR



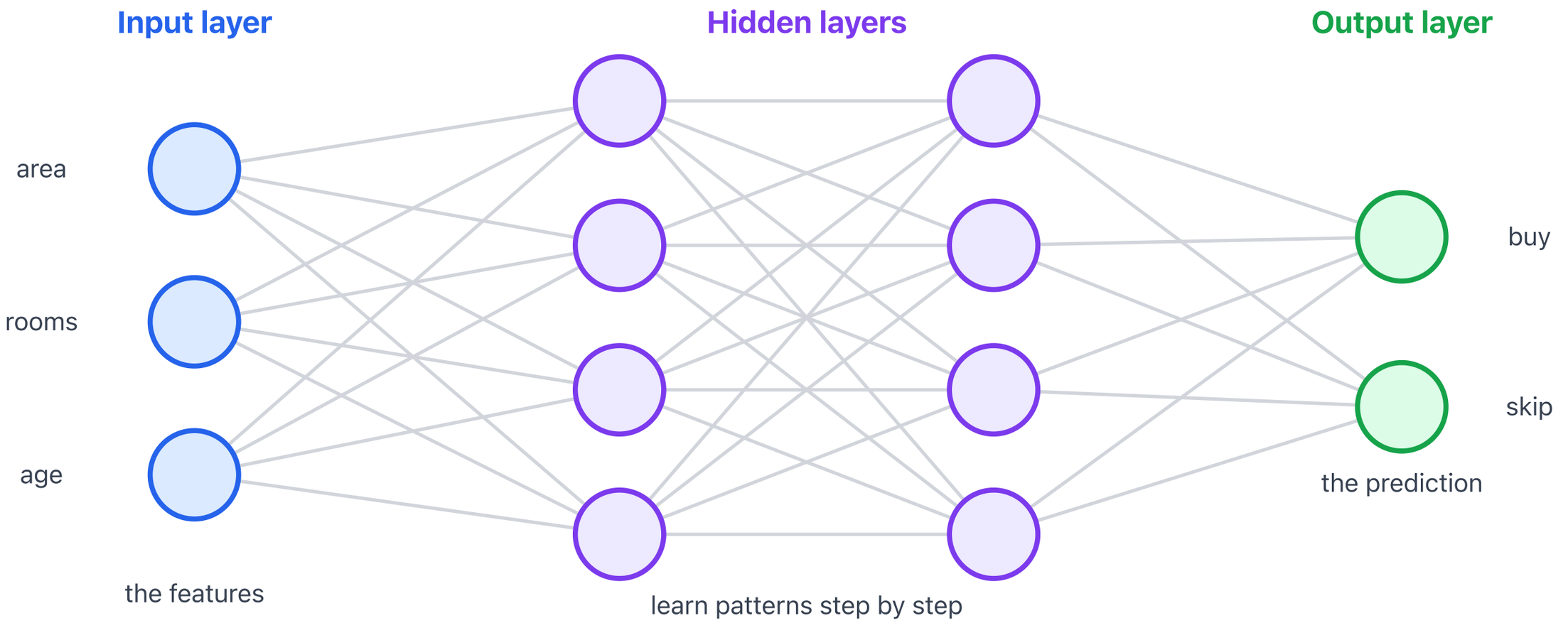
XOR: output 1 when inputs are different.

● = 1 ● = 0

No single straight line can separate red from blue!

> **In plain English:** One neuron can only draw one straight line. To solve harder problems, **stack neurons into layers** → a neural network.

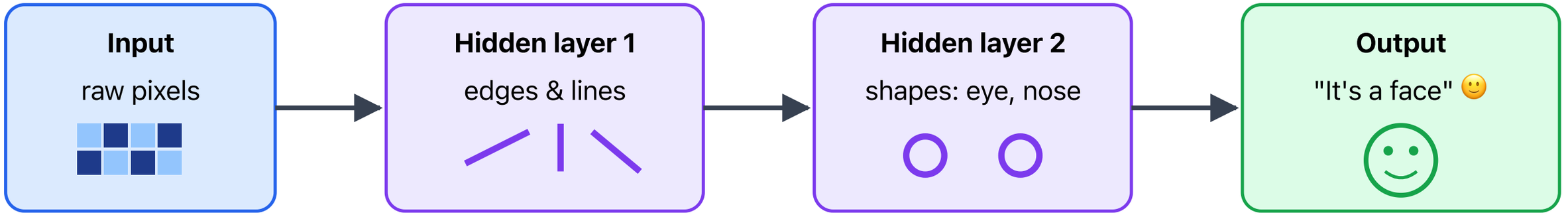
A Neural Network = Layers of Neurons



In plain English: Early layers find simple patterns; later layers combine them into complex ones (edges → shapes → faces). "Deep learning" = many hidden layers.

What Do Hidden Layers Actually Do?

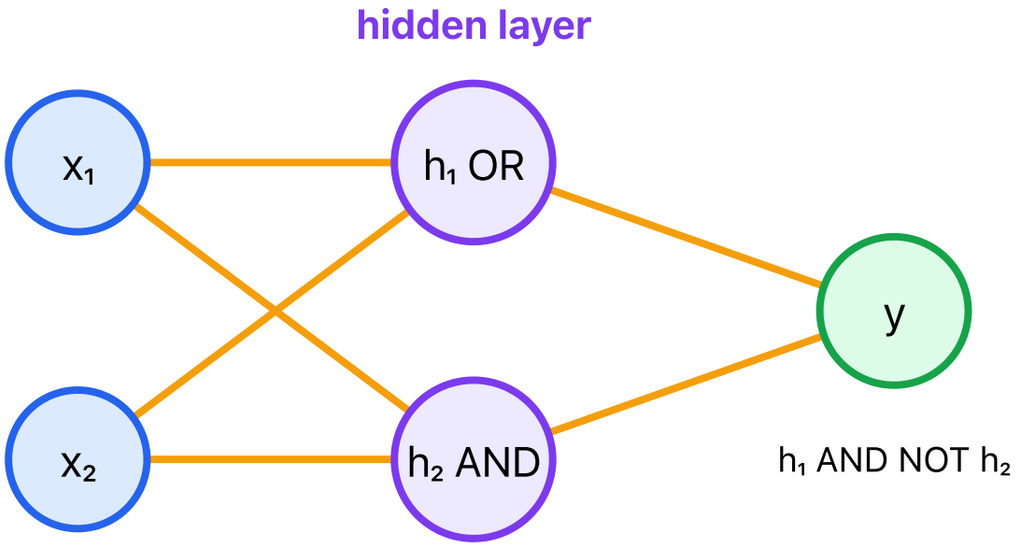
They're called **hidden** because we never see their values in the data. We only see the inputs and the final output. Each hidden layer **turns its inputs into new, more useful features**.



simple patterns → combined patterns → the answer

🏦 Fraud example: one hidden neuron may learn "large amount + new merchant", another "transaction at 3 a.m. from a new country". The output layer combines these clues into a fraud score.

Hidden Layers in Action: Solving XOR



all neurons use the step function

- $h_1 = \text{step}(x_1 + x_2 - 0.5)$ → "at least one is on" (OR)
- $h_2 = \text{step}(x_1 + x_2 - 1.5)$ → "both are on" (AND)
- $y = \text{step}(h_1 - h_2 - 0.5)$ → "OR but not AND"

x_1	x_2	h_1	h_2	y
0	0	0	0	0
0	1	1	0	**1**
1	0	1	0	**1**
1	1	1	1	0

In plain English: The hidden layer breaks a hard problem into two easy ones (OR, AND), and the output neuron combines them. That's the power of hidden layers.

How Many Hidden Layers and Neurons?

↔️ Width = neurons per layer

More neurons → more patterns detected at that level.

↕️ Depth = number of layers

More layers → more levels of combining (patterns of patterns).

Universal approximation theorem: even one hidden layer with enough neurons can approximate almost any function. In practice, deeper networks learn the same thing with far fewer neurons.

📏 Practical starting points

- Simple tabular data: **1–2 hidden layers**, 32–128 neurons each
- Often use powers of 2 (32, 64, 128) and a **funnel** shape, e.g. 128 → 64 → 32
- Images, speech, text: dozens to hundreds of layers (CNNs, Transformers)
- Start small. Grow if you underfit, shrink or regularise if you overfit

In plain English: The number of hidden layers and neurons are **hyperparameters**: you choose them and then check the results on the validation set.

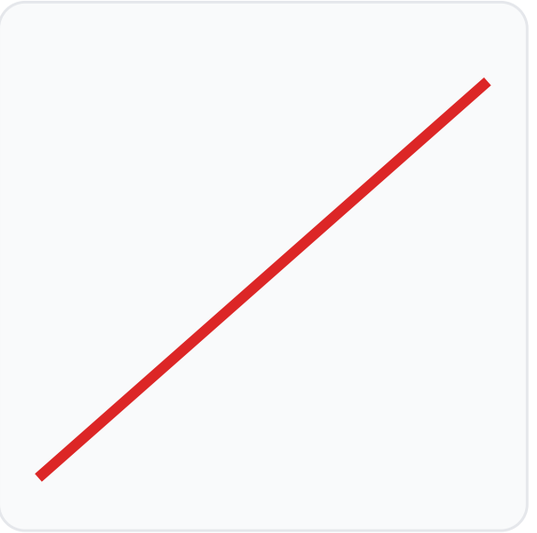
Why Do We Need Activation Functions?

Without them, two layers collapse into one:

$$W_2(W_1x + b_1) + b_2 = \underbrace{(W_2W_1)}_{W'}x + \underbrace{(W_2b_1 + b_2)}_{b'}$$

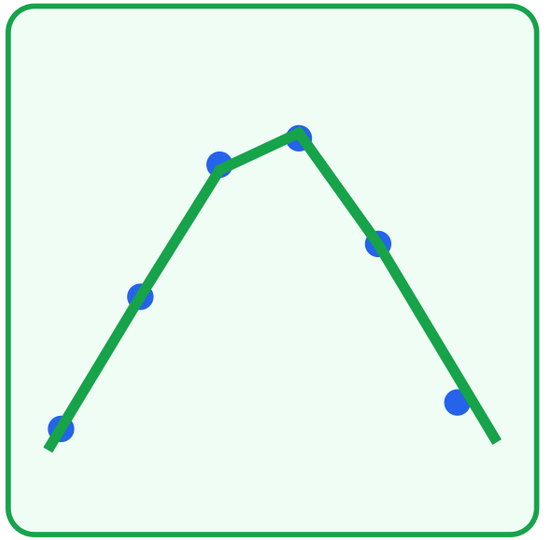
Still just a straight line, no matter how many layers you stack! 😕

With a non-linear activation between layers, each neuron adds a "bend". Lots of bends combined can trace any curve.



no activation

always a straight line

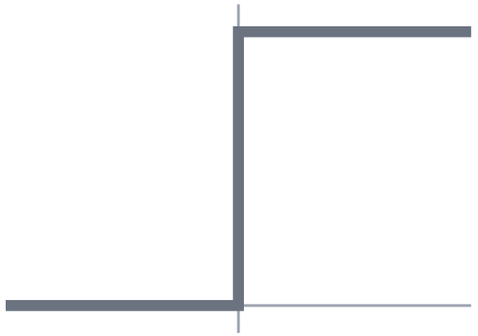


with ReLU

straight pieces join into a curve

In plain English: The activation function is the neuron's "decision": how strongly should I fire for this input? It's what lets networks learn curvy, real-world patterns.

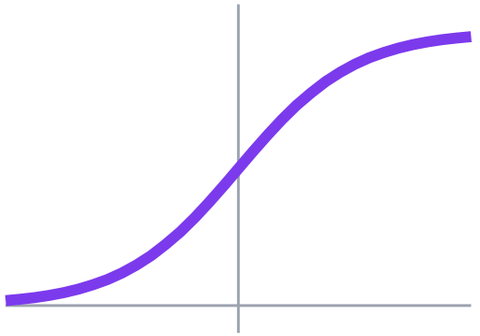
Activation Functions at a Glance



Step

0 or 1

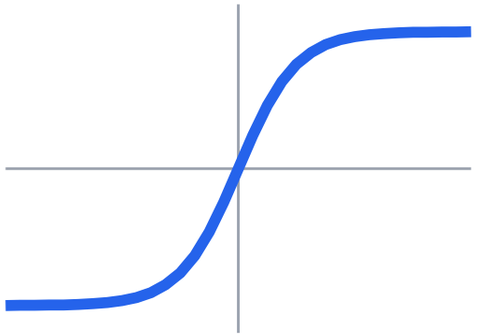
perceptron



Sigmoid

0 to 1

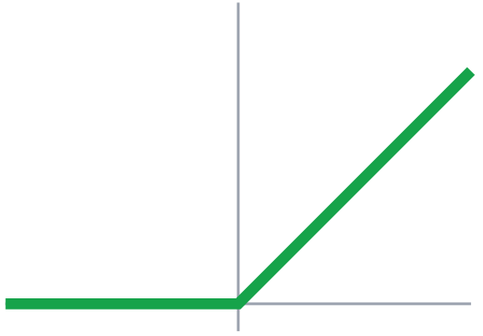
output layer for yes/no



Tanh

–1 to 1

centred at zero



ReLU $\max(0, z)$

simple & fast

default for hidden layers ⭐

Plus **softmax**, which works on a whole layer at once rather than one neuron. Let's look at each one.

Sigmoid — "How likely is yes?"

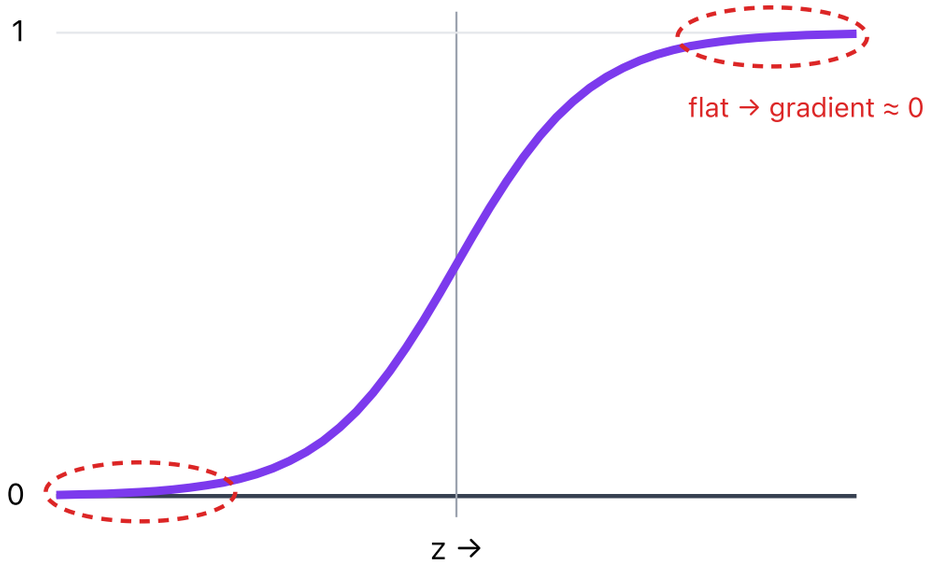
$$\sigma(z) = \frac{1}{1 + e^{-z}} \qquad \text{range: } (0,\ 1)$$

z	−5	−2	0	2	5
$\sigma(z)$	0.007	0.12	0.5	0.88	0.993

✅ Output reads as a **probability**

✅ Use in the **output layer** for binary & multilabel classification

❌ **Vanishing gradient**: its slope is at most 0.25 and almost 0 at the ends. Over 5 layers: $0.25^5 \approx 0.001$, so early layers barely learn

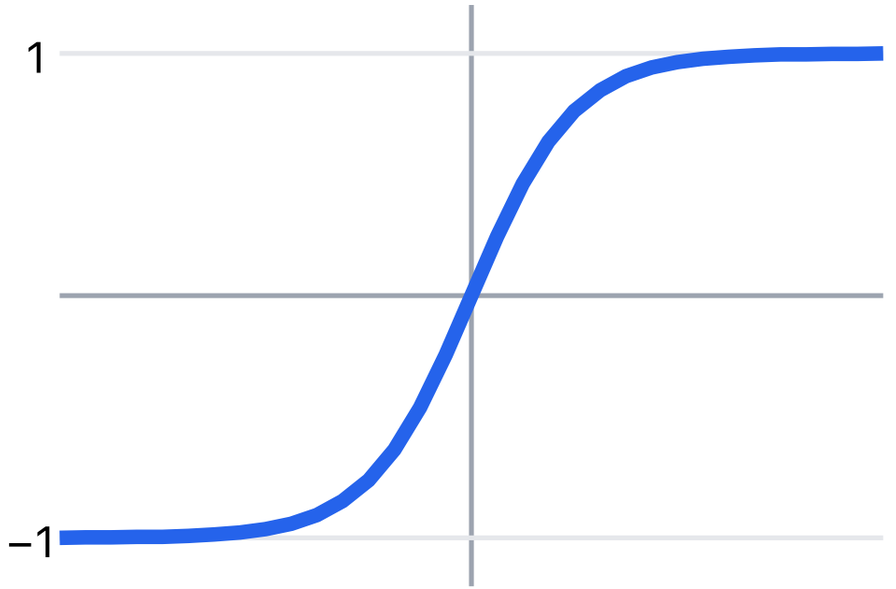


In plain English: Sigmoid squashes any number into 0–1, like a dimmer switch. Great for a final "yes/no probability", but too slow to learn with inside deep networks.

Tanh — Sigmoid's Zero-Centred Cousin

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \qquad \text{range: } (-1,\ 1)$$

z	−2	−1	0	1	2
$\tanh(z)$	−0.96	−0.76	0	0.76	0.96



✅ Centred at 0 (outputs are both negative and positive), so it usually trains faster than sigmoid

✅ Common in **recurrent networks** (LSTM, GRU)

❌ Still flattens at the ends, so gradients still vanish

> **In plain English:** Same S-shape as sigmoid, but stretched from −1 to +1, so a neuron can say "strongly no", "neutral" or "strongly yes".

ReLU — The Default for Hidden Layers ⭐

$$\text{ReLU}(z) = \max(0,\ z) \qquad \text{range: } [0,\ \infty)$$

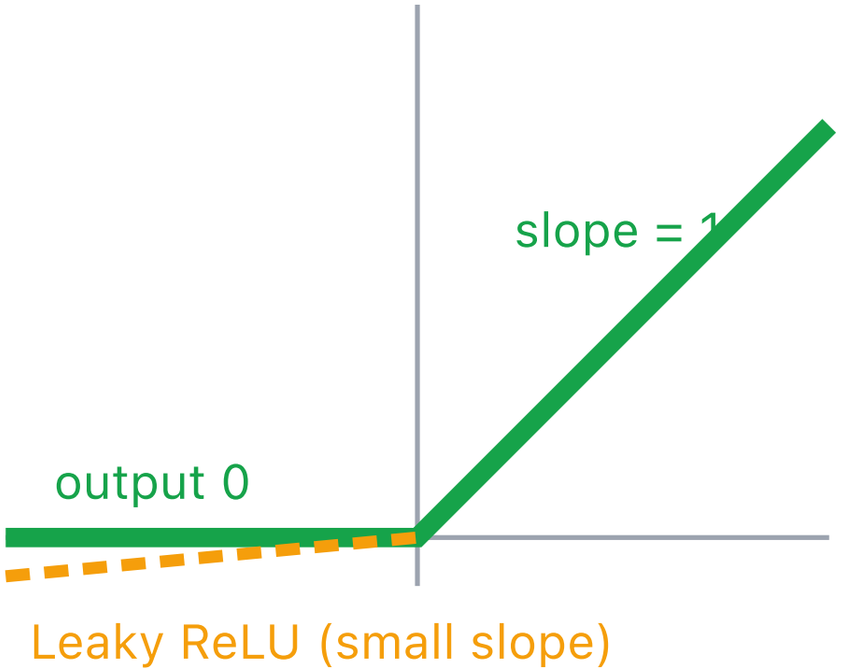
z	−3	−1	0	2	5
ReLU	0	0	0	2	5

✅ Very fast — just "if negative, make it 0"

✅ Slope is exactly 1 for positive inputs, so **gradients don't vanish** and deep networks can train

❌ **"Dying ReLU"**: a neuron stuck in the negative zone outputs 0 forever and stops learning

🔧 Fix: **Leaky ReLU** $= \max(0.01z,\ z)$ keeps a small slope for negatives. GELU / SiLU are smooth variants used in Transformers

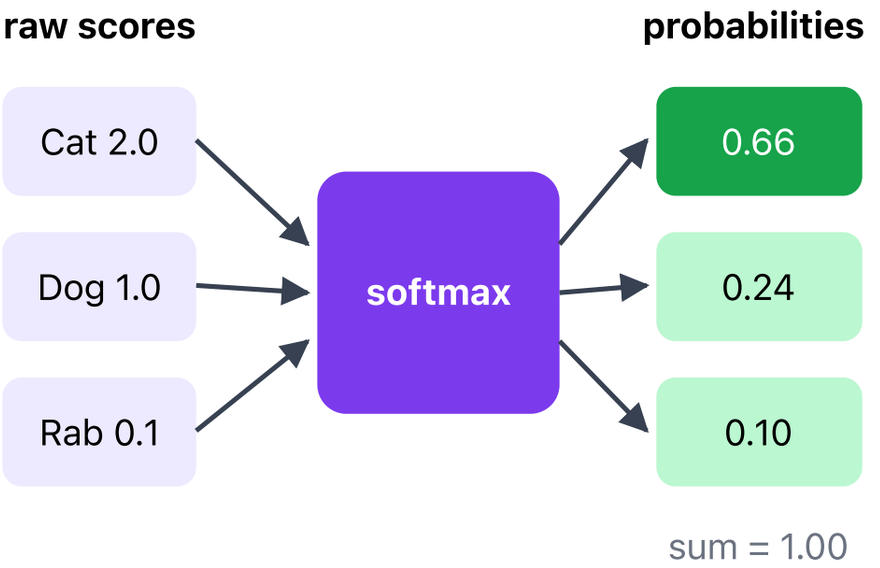


> **In plain English:** ReLU is a one-way gate: negative signals are blocked and positive signals pass through unchanged. Simple, fast, and it works, so it's the first choice for hidden layers.

Softmax — Picking One Class Out of Many

$$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

- Applied to the **whole output layer**, not to one neuron
- Every output is between 0 and 1, and they **add up to 1**
- e^z makes the largest score stand out even more ("soft" version of picking the max)
- Use in the **output layer for multiclass** problems, paired with categorical cross-entropy



In plain English: Softmax is a vote where the classes share 100% of the probability. Raising one class's share must lower the others.

Cheat Sheet: Which Activation Goes Where?

Where	Activation	Why
Hidden layers	**ReLU** (or Leaky ReLU / GELU)	fast, avoids vanishing gradients
Hidden layers in RNNs	tanh (and sigmoid gates)	bounded, zero-centred
Output: regression	**none** (linear)	output can be any number
Output: binary	**sigmoid**	one probability of "yes"
Output: multiclass	**softmax**	probabilities over K classes that sum to 1
Output: multilabel	**sigmoid** on each neuron	independent yes/no per label

> **In plain English:** Use ReLU inside the network. Pick the output activation to match the kind of answer you want.

Counting Parameters

Every connection has a weight; every neuron (except inputs) has a bias. For each layer:

$$\text{parameters} = (\text{inputs} \times \text{neurons}) + \text{neurons}$$

Small network: 3 → 4 → 2

- Layer 1: $3 \times 4 + 4 = 16$
- Layer 2: $4 \times 2 + 2 = 10$
- **Total = 26 parameters**

Digit recogniser: 784 → 128 → 10

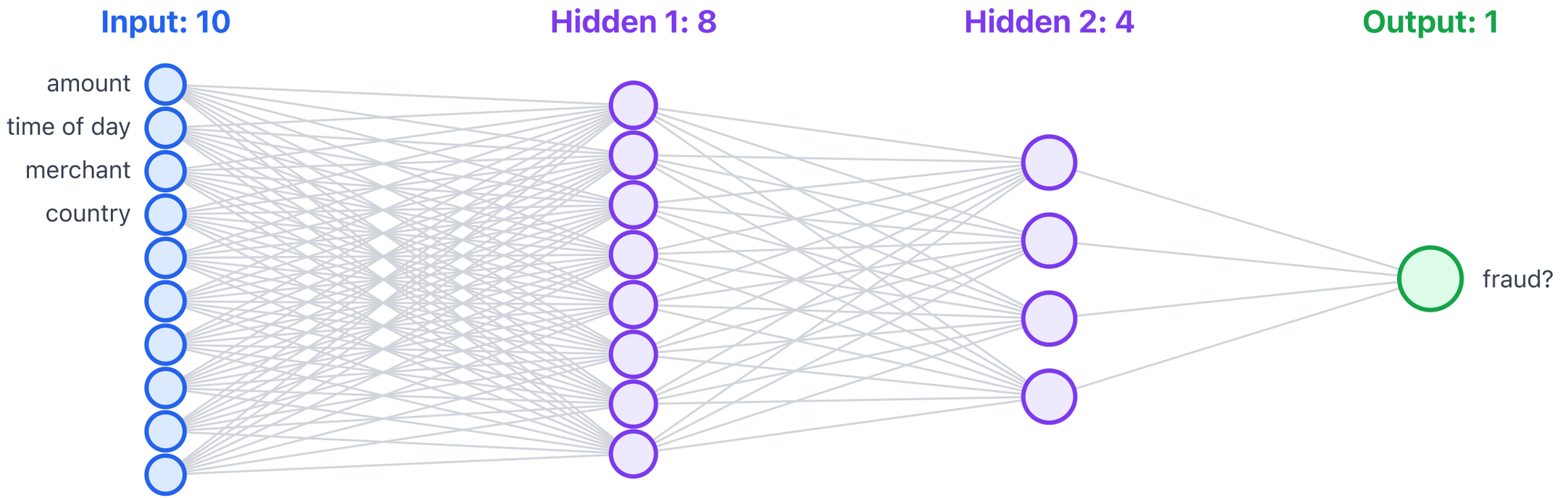
(28×28-pixel image = 784 inputs)

- Layer 1: $784 \times 128 + 128 = 100{,}480$
- Layer 2: $128 \times 10 + 10 = 1{,}290$
- **Total = 101,770 parameters**

For scale: modern large language models have **billions** of parameters. More parameters = more capacity to learn, but more data & compute needed and a higher risk of overfitting.

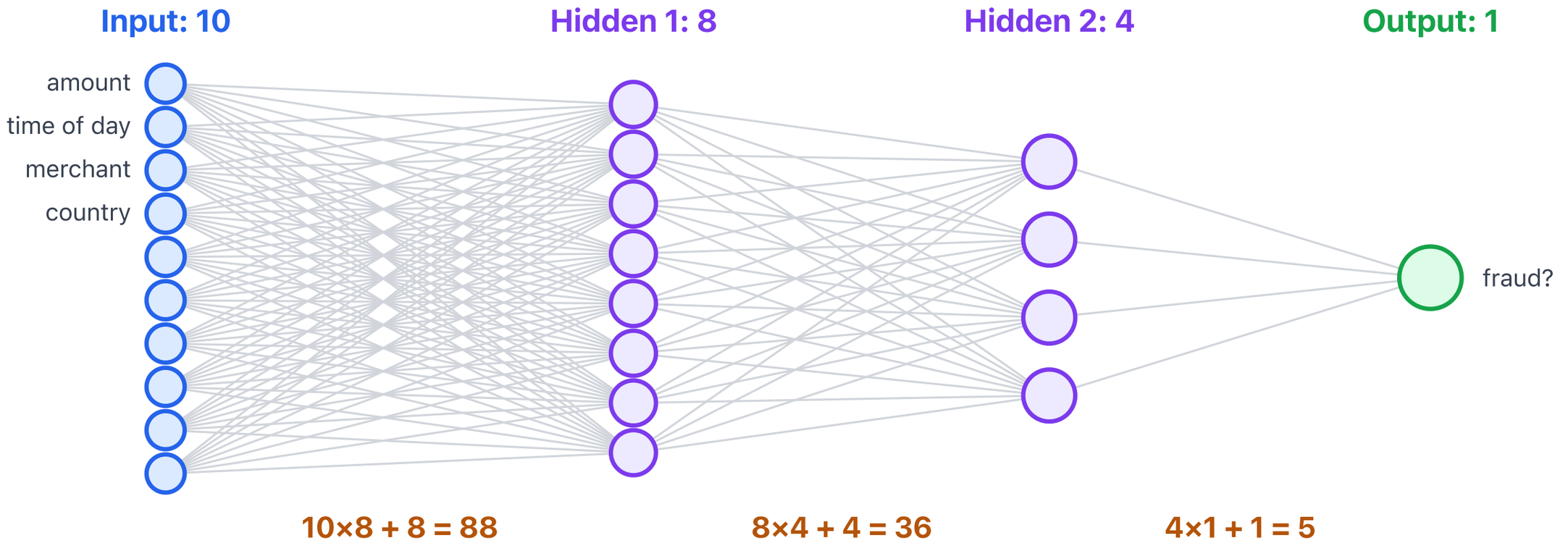
🤔 Your Turn: Count the Parameters

A bank's **fraud detector** for card transactions has two hidden layers and a sigmoid output (probability of fraud). **How many parameters does it learn?**



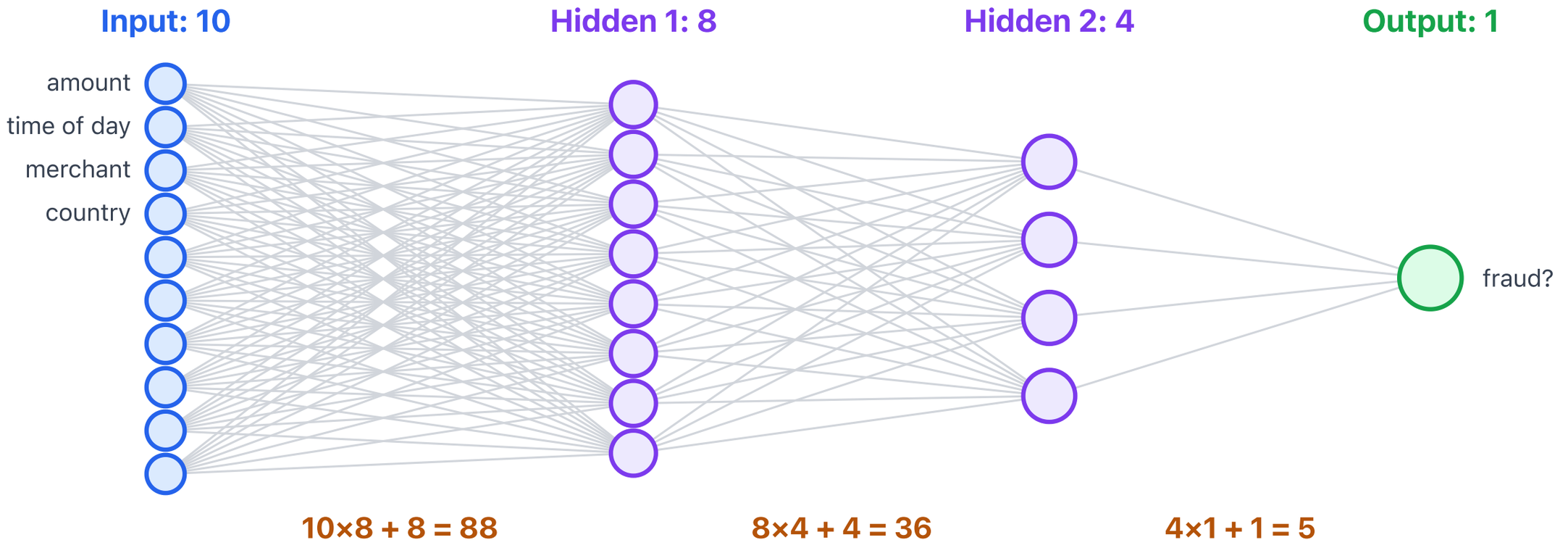
🤔 Your Turn: Count the Parameters

A bank's **fraud detector** for card transactions has two hidden layers and a sigmoid output (probability of fraud). **How many parameters does it learn?**



🤔 Your Turn: Count the Parameters

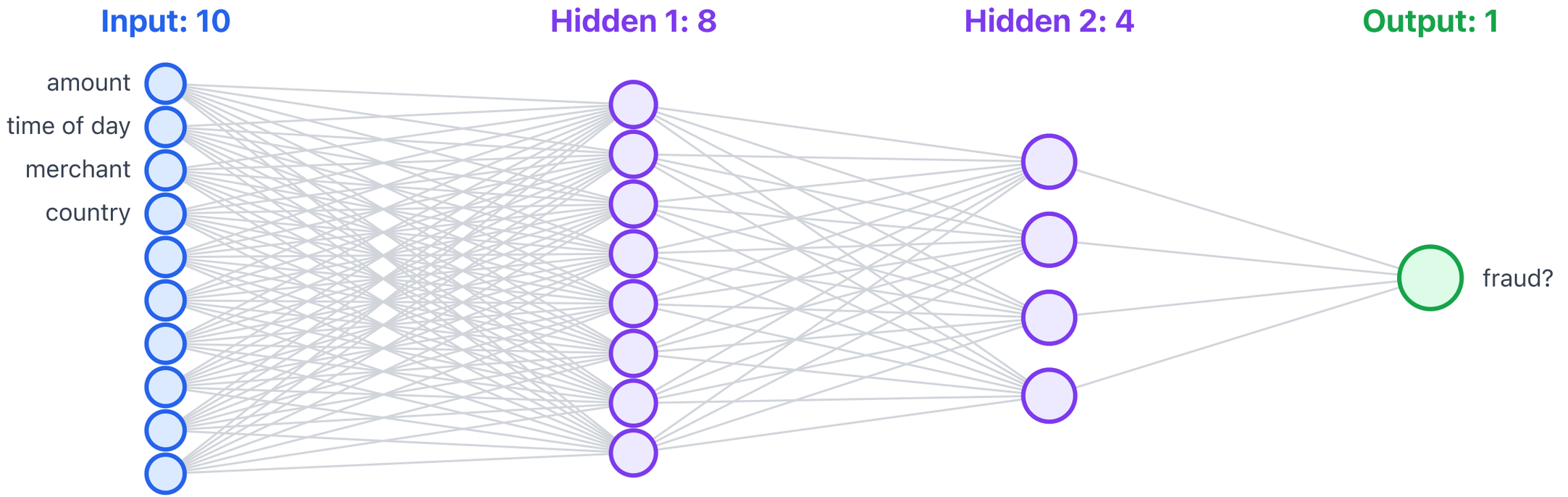
A bank's **fraud detector** for card transactions has two hidden layers and a sigmoid output (probability of fraud). **How many parameters does it learn?**



Total = 88 + 36 + 5 = 129 parameters

🤔 Your Turn: Count the Parameters

A bank's **fraud detector** for card transactions has two hidden layers and a sigmoid output (probability of fraud). **How many parameters does it learn?**



Total = 88 + 36 + 5 = 129 parameters

Forgot the biases? Then you get $80 + 32 + 4 = 116$.
Each non-input neuron adds one bias: $+13$.

Parameters vs Hyperparameters

Parameters — learned by the model

- Weights w
- Biases b

Found automatically during training.

🍳 Like the **taste** a dish ends up with.

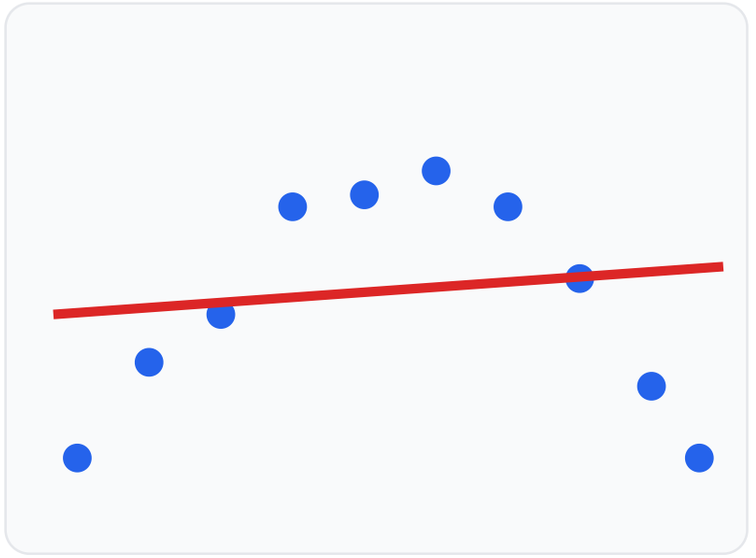
Hyperparameters — chosen by you

- Learning rate η
- Number of layers & neurons per layer
- Activation function
- Batch size, number of epochs
- Optimizer (SGD, Adam…)
- Dropout rate, regularisation strength

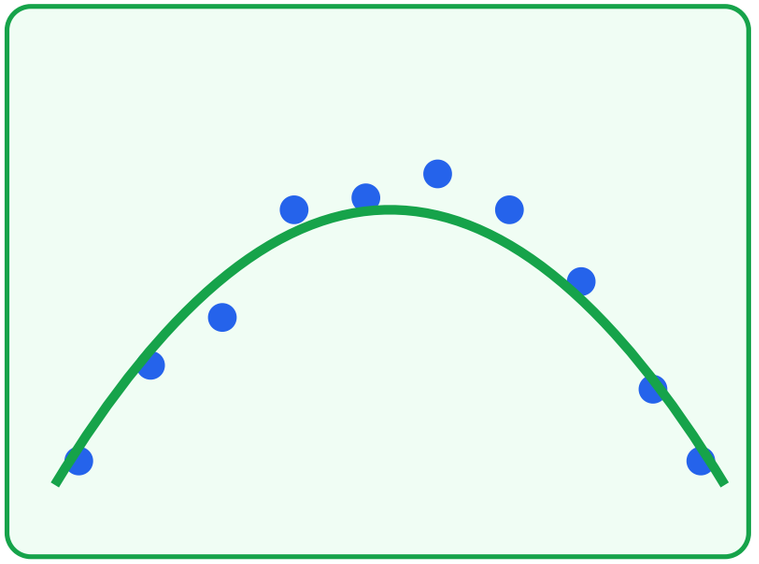
🍳 Like the **oven temperature & cooking time**.

Epoch = one full pass over the training data. **Batch size** = how many examples are used for each weight update.

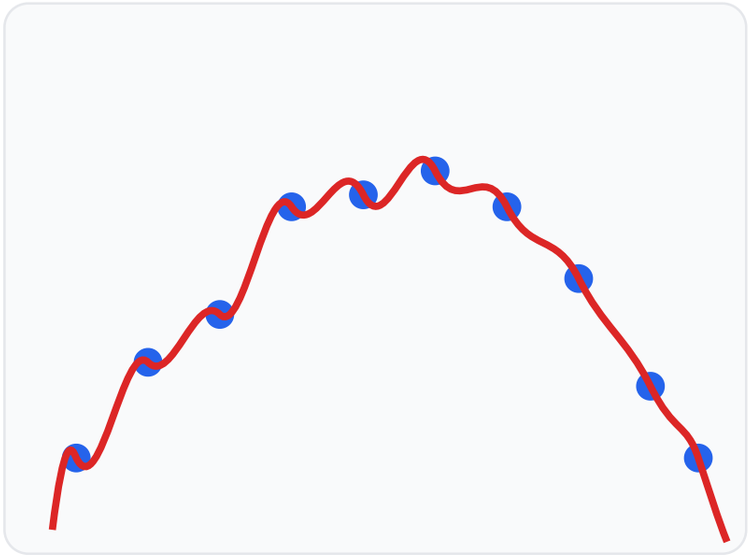
Underfitting vs Overfitting



Underfitting
too simple — misses the pattern



Just right ✓
captures the real trend



Overfitting
memorises noise — fails on new data

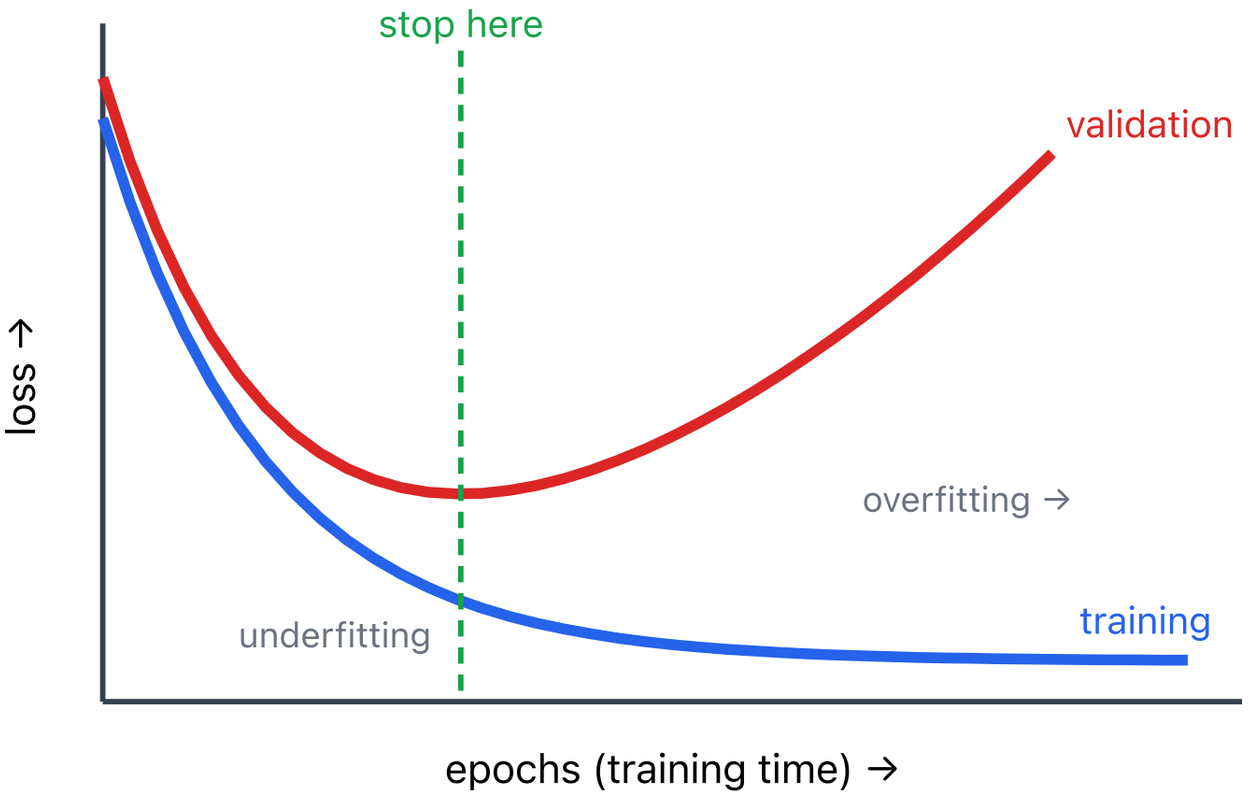
In plain English: Underfitting = a student who didn't study. Overfitting = a student who memorised past papers word-for-word but can't handle a new question.

Spotting Overfitting: Training vs Validation Loss

Hold back a **validation set** the model never trains on.

- Training loss keeps falling
- Validation loss falls, then **rises** → the model has started memorising

In plain English: Early stopping: stop training (and keep the weights) at the point where validation loss is lowest.



Tuning a Neural Network

🔧 Fix underfitting (too simple)

- Add more layers or neurons
- Train for more epochs
- Increase the learning rate a little
- Add better features

🛡️ Fix overfitting (memorising)

- Get more training data
- **Dropout** — randomly switch off neurons while training
- **L2 regularisation** — penalise large weights
- **Early stopping**
- Use a smaller network

Searching for good hyperparameters: Grid search tries every combination; random search samples combinations (often faster); tools like Optuna automate this.

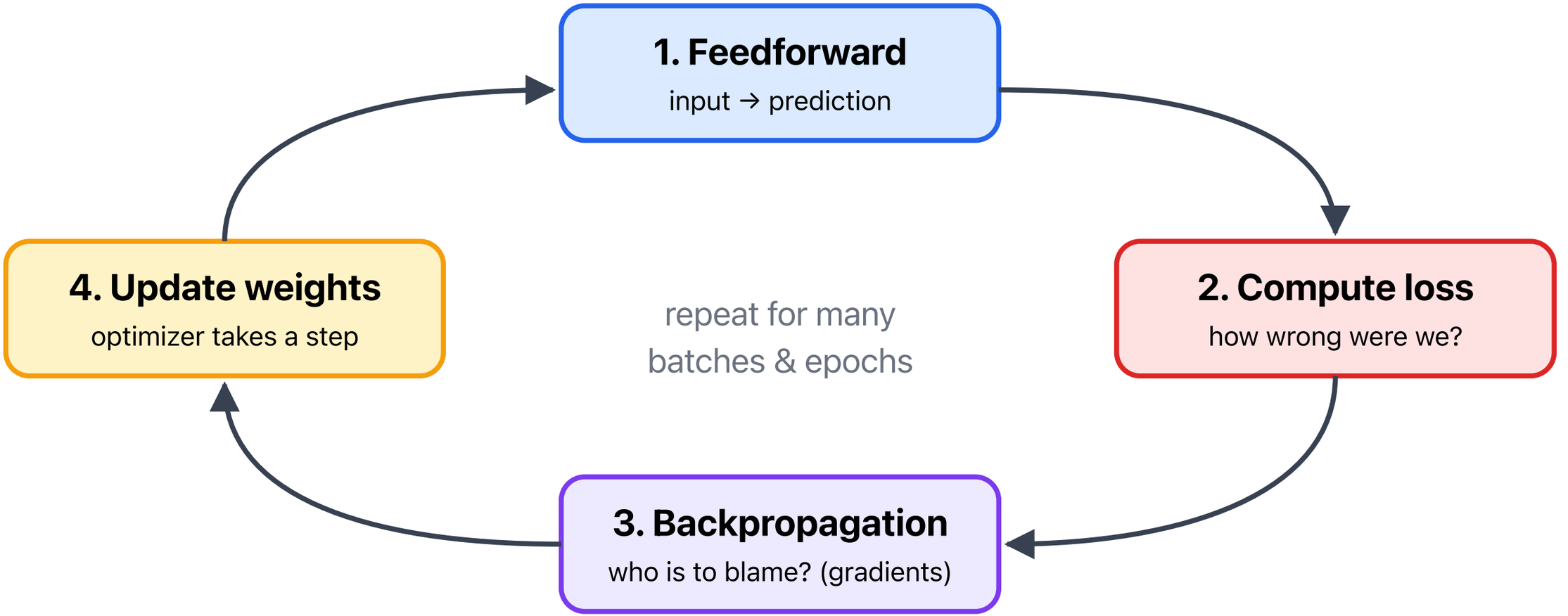
In plain English: Tune the learning rate first — it matters most. Change one thing at a time and always judge on the **validation set**.

07

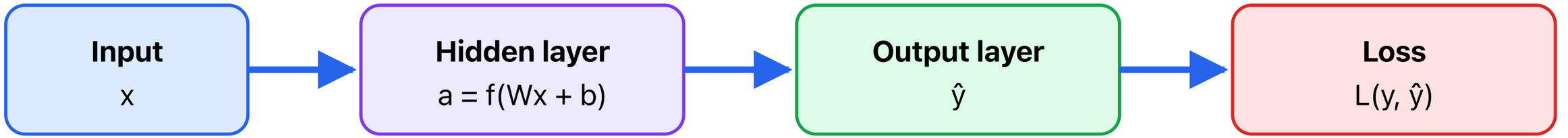
Feedforward & Backpropagation

How a network makes predictions — and how it learns from mistakes

The Training Loop



Feedforward: Data Flows Forward



→ → → left to right, layer by layer → → →

Each layer l does the same neuron computation on the previous layer's outputs:

$$\mathbf{a}^{(l)} = f\left(\mathbf{W}^{(l)}\,\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}\right)$$

In plain English: Feedforward is simply "make a prediction": push the inputs through each layer in turn until an answer comes out the other end.

Worked Example — Forward Pass

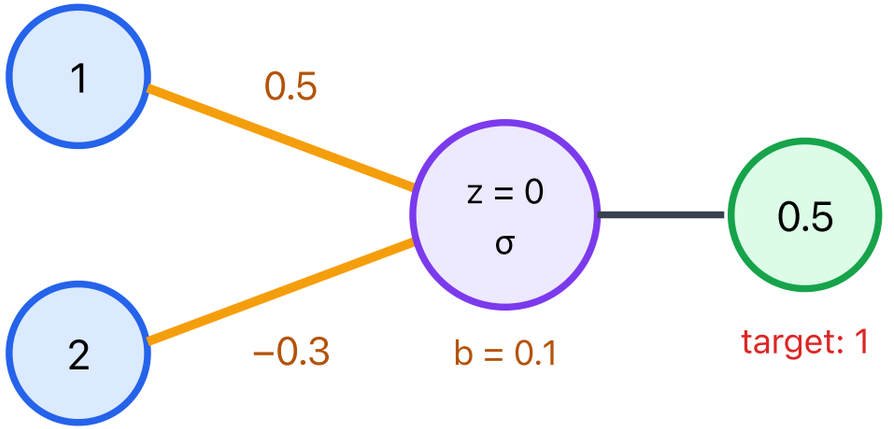
One sigmoid neuron. Inputs $x_1 = 1,\ x_2 = 2$; weights $w_1 = 0.5,\ w_2 = -0.3$; bias $b = 0.1$. True answer $y = 1$.

$$z = 0.5(1) + (-0.3)(2) + 0.1 = 0$$

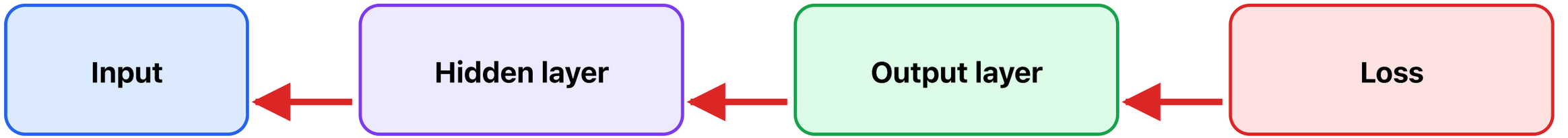
$$\hat{y} = \sigma(0) = 0.5$$

$$L = (y - \hat{y})^2 = (1 - 0.5)^2 = 0.25$$

In plain English: The model is only 50% sure when the answer is definitely 1. We need to adjust the weights.



Backpropagation: Assigning the Blame



← ← ← error signal flows backwards ← ← ←

- Start at the loss and work **backwards** through each layer
- For every weight, compute its **gradient** $\frac{\partial L}{\partial w}$: "if I nudge this weight up a little, how much does the loss change?"
- Positive gradient → lower the weight; negative gradient → raise it

> **In plain English:** It's like a team reviewing a lost match: the error is traced back so each player (weight) learns how much they contributed and how to adjust.

The Chain Rule — the Engine of Backprop

The loss depends on w through a chain: $w \to z \to \hat{y} \to L$. So we multiply the effect of each link:

$$\frac{\partial L}{\partial w_1} = \underbrace{\frac{\partial L}{\partial \hat{y}}}_{\text{loss vs output}} \cdot \underbrace{\frac{\partial \hat{y}}{\partial z}}_{\text{activation slope}} \cdot \underbrace{\frac{\partial z}{\partial w_1}}_{\text{input } x_1}$$

Link	Formula	Our numbers
$\partial L/\partial \hat{y}$	$-2(y-\hat{y})$	$-2(1-0.5) = -1$
$\partial \hat{y}/\partial z$	$\sigma(z)(1-\sigma(z))$	$0.5 \times 0.5 = 0.25$
$\partial z/\partial w_1$	x_1	1

In plain English: Gears: if gear A turns gear B twice as fast, and B turns C three times as fast, then A turns C 2 × 3 = 6 times as fast. Backprop just multiplies these "speeds".

Worked Example — Backward Pass & Update

Gradients (from the chain rule):

$$\frac{\partial L}{\partial w_1} = -1 \times 0.25 \times 1 = -0.25$$

$$\frac{\partial L}{\partial w_2} = -1 \times 0.25 \times 2 = -0.5$$

$$\frac{\partial L}{\partial b} = -1 \times 0.25 \times 1 = -0.25$$

Update with learning rate $\eta = 0.1$:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

- $w_1 = 0.5 - 0.1(-0.25) = 0.525$
- $w_2 = -0.3 - 0.1(-0.5) = -0.25$
- $b = 0.1 - 0.1(-0.25) = 0.125$

Worked Example — Backward Pass & Update

Gradients (from the chain rule):

$$\frac{\partial L}{\partial w_1} = -1 \times 0.25 \times 1 = -0.25$$

$$\frac{\partial L}{\partial w_2} = -1 \times 0.25 \times 2 = -0.5$$

$$\frac{\partial L}{\partial b} = -1 \times 0.25 \times 1 = -0.25$$

Update with learning rate $\eta = 0.1$:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

- $w_1 = 0.5 - 0.1(-0.25) = 0.525$
- $w_2 = -0.3 - 0.1(-0.5) = -0.25$
- $b = 0.1 - 0.1(-0.25) = 0.125$

Forward pass again: $z = 0.15$, $\hat{y} = 0.537$, loss $= (1 - 0.537)^2 = 0.214$ (it was 0.25) 🎉

Worked Example — Backward Pass & Update

Gradients (from the chain rule):

$$\frac{\partial L}{\partial w_1} = -1 \times 0.25 \times 1 = -0.25$$

$$\frac{\partial L}{\partial w_2} = -1 \times 0.25 \times 2 = -0.5$$

$$\frac{\partial L}{\partial b} = -1 \times 0.25 \times 1 = -0.25$$

Update with learning rate $\eta = 0.1$:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

- $w_1 = 0.5 - 0.1(-0.25) = 0.525$
- $w_2 = -0.3 - 0.1(-0.5) = -0.25$
- $b = 0.1 - 0.1(-0.25) = 0.125$

Forward pass again: $z = 0.15$, $\hat{y} = 0.537$, loss $= (1 - 0.537)^2 = 0.214$ (it was 0.25) 🎉

> **In plain English:** One step made the model a little better. Repeat this thousands of times over many examples and the network learns.

Which Loss Function Should I Use?

Problem	Output layer	Loss function	Example
Regression	1 neuron, no activation (linear)	**MSE** (or MAE / Huber if there are outliers)	House price
Binary classification	1 neuron, sigmoid	**Binary cross-entropy**	Fraud or not
Multiclass	K neurons, softmax	**Categorical cross-entropy**	Digit 0–9
Multilabel	K neurons, sigmoid each	**Binary cross-entropy** per label	Movie genres

> Why not MSE for classification? Cross-entropy gives **much bigger gradients** when the model is confidently wrong, so it learns faster and doesn't get stuck.

08

Optimization

Gradient descent, SGD, Momentum, RMSProp, Adam

Gradient Descent: Walking Downhill Blindfolded

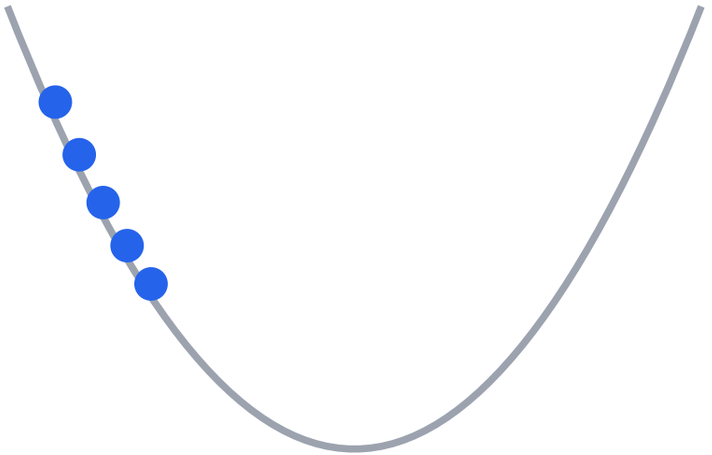
You're on a foggy mountain and want to reach the valley. You can only feel the slope under your feet, so you take a step in the **steepest downhill direction**, again and again.

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

η = learning rate (step size) · gradient = slope

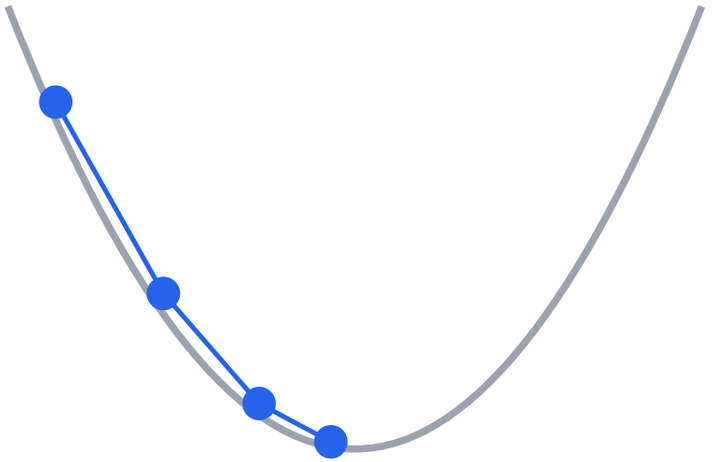


The Learning Rate Matters



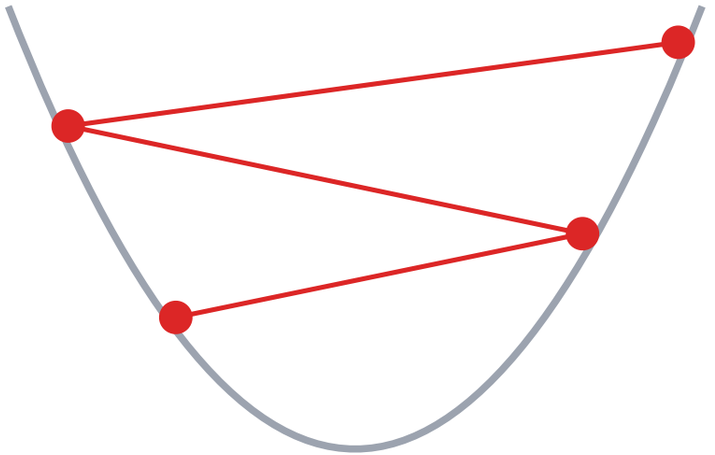
Too small 🐢

takes forever to arrive



Just right ✓

steady, quick convergence



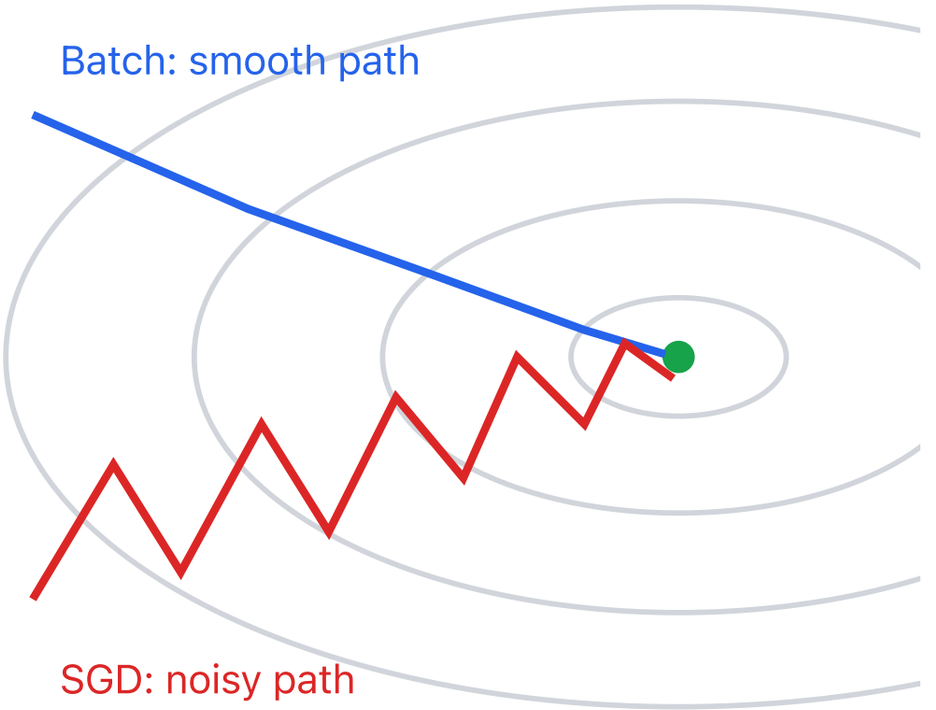
Too large 🚀

overshoots and bounces away

In plain English: Typical starting values: 0.1 for SGD, 0.001 for Adam. If the loss explodes, lower it; if it barely moves, raise it.

Batch GD vs SGD vs Mini-batch

Method	Examples per update	Behaviour
Batch GD	all of them	smooth but slow; needs lots of memory
Stochastic GD (SGD)	1	very fast, very noisy (zig-zags)
Mini-batch ⭐	e.g. 32–256	best of both — the standard choice



In plain English: In practice, when people say "SGD" they usually mean **mini-batch** SGD. The noise can even help escape bad spots.

Momentum: A Ball Rolling Downhill

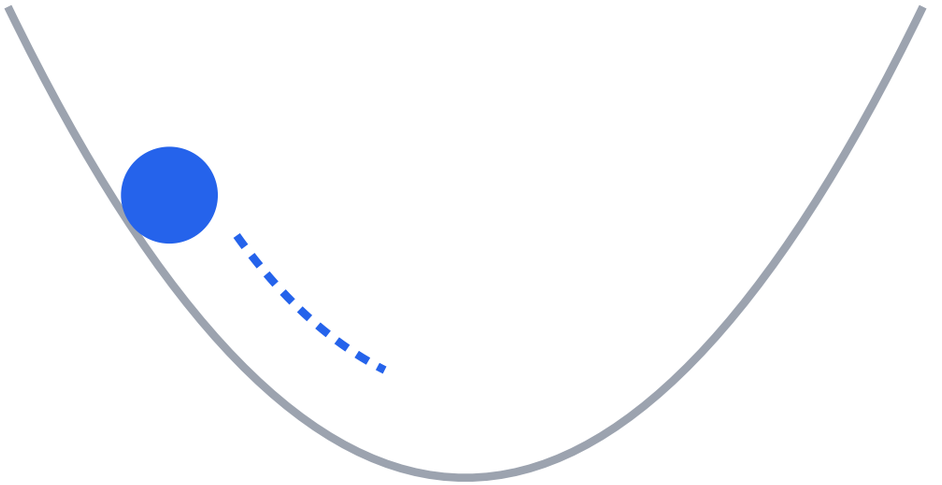
Plain SGD forgets the past at every step. **Momentum** keeps a running "velocity" — an average of past gradients:

$$v \leftarrow \beta\, v + (1 - \beta)\, g$$

$$w \leftarrow w - \eta\, v$$

g = current gradient, $\beta \approx 0.9$ = how much of the past to remember

> **In plain English:** Like a heavy ball: it builds speed in consistent directions and smooths out the zig-zags.



gains speed on long slopes

RMSProp: A Custom Step Size for Every Weight

$$s \leftarrow \beta\, s + (1-\beta)\, g^2 \qquad\qquad w \leftarrow w - \frac{\eta}{\sqrt{s}+\epsilon}\, g$$

- s tracks a running average of **squared gradients** — how big this weight's gradients usually are
- Weights with **big, jumpy** gradients → divide by a big number → **smaller steps**
- Weights with **tiny** gradients → divide by a small number → **bigger steps**
- $\epsilon \approx 10^{-8}$ just prevents dividing by zero; $\beta \approx 0.9$

> **In plain English:** Like adjusting your stride to the terrain: careful, small steps on steep, rocky ground; long strides on flat ground.

Adam = Momentum + RMSProp

$$m \leftarrow \beta_1 m + (1 - \beta_1)\, g \quad \text{(momentum: direction)}$$

$$v \leftarrow \beta_2 v + (1 - \beta_2)\, g^2 \quad \text{(RMSProp: scale)}$$

$$\hat{m} = \frac{m}{1 - \beta_1^t}, \quad \hat{v} = \frac{v}{1 - \beta_2^t} \quad \text{(bias correction)}$$

$$w \leftarrow w - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$

Default settings (work well almost always)

- $\eta = 0.001$
- $\beta_1 = 0.9$ — memory for gradients
- $\beta_2 = 0.999$ — memory for squared gradients
- $\epsilon = 10^{-8}$

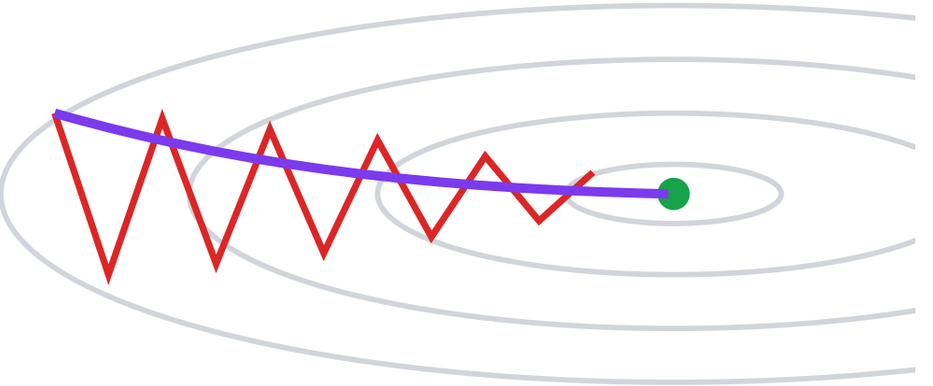
Bias correction: m and v start at 0, so early averages are too small; dividing by $(1 - \beta^t)$ fixes this for the first few steps t.

In plain English: Adam knows **which way** to go (momentum) and **how big a step** each weight should take (RMSProp). It's the go-to default optimizer.

Optimizers Compared

Optimizer	Key idea	When to use
SGD	step against the gradient	simple baseline; with tuning can generalise very well
SGD + Momentum	remember past direction	computer vision (CNNs) — popular with a learning-rate schedule
RMSProp	per-weight step size	recurrent networks, noisy problems
Adam ⭐	momentum + per-weight step	default first choice — fast, little tuning

SGD: zig-zags across the valley



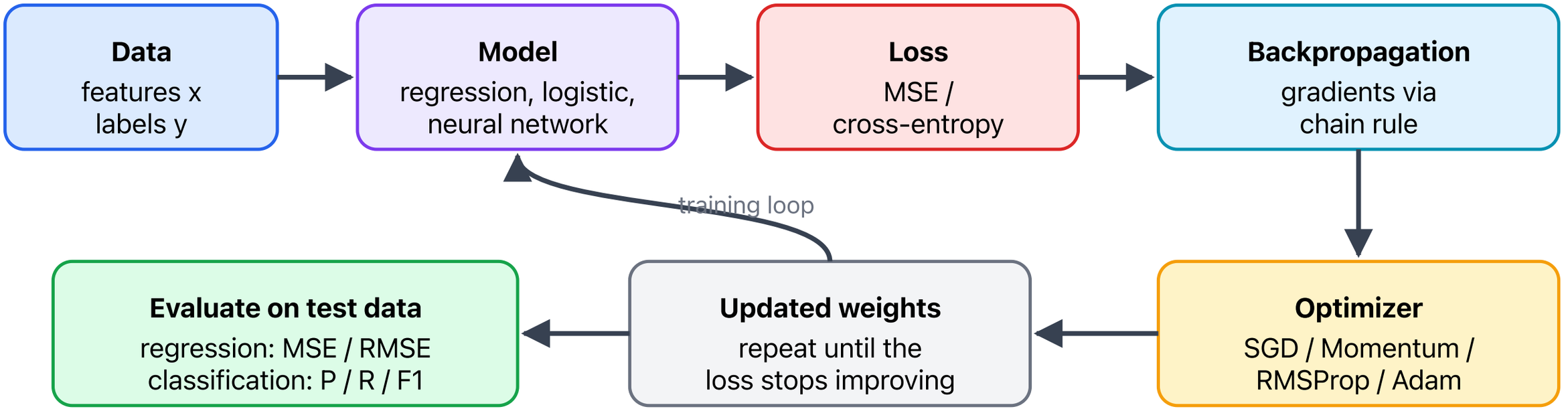
Adam: heads straight to the minimum

> Start with **Adam (lr = 0.001)**. If you need the last bit of accuracy and have time to tune, try SGD + momentum with a learning-rate schedule.



Putting It All Together

The Big Picture



Hyperparameters (learning rate, layers, batch size, epochs…) are tuned using a validation set.

Glossary

- **Feature** — an input variable (x)
- **Label** — the answer to predict (y)
- **Regression** — predict a number
- **Classification** — predict a category
- **Loss** — how wrong the model is
- **Weight / bias** — learned parameters
- **Hidden layer** — layer between input and output that learns new features
- **Activation** — adds non-linearity (ReLU, sigmoid)
- **Softmax** — scores → probabilities that sum to 1
- **Epoch** — one pass over all training data
- **Batch** — examples per weight update
- **Gradient** — slope of the loss for a weight
- **Backprop** — computing gradients backwards
- **Learning rate** — step size
- **Overfitting** — memorising instead of learning
- **Precision** — trust in a "yes"
- **Recall** — share of real "yes" cases found

Next Steps

- 🐍 Try **scikit-learn**: `LinearRegression`, `LogisticRegression`, `classification_report`
- 🔥 Build a small network in **PyTorch** or **Keras** on the MNIST digits dataset
- 🎮 Play with the TensorFlow Playground to see neurons and layers learn live
- 📚 Watch 3Blue1Brown — Neural Networks for beautiful visual intuition

> **In plain English:** Every ML model follows the same recipe: **predict → measure the loss → compute gradients → update → repeat**.

Thank you!

Questions?